

Licence IIDL / S5 :
Ingénierie Informatique et Développement Logiciel

Développement d'applications en Java

Architectures des Applications Distribuées

PR. ABDOUN OTMAN

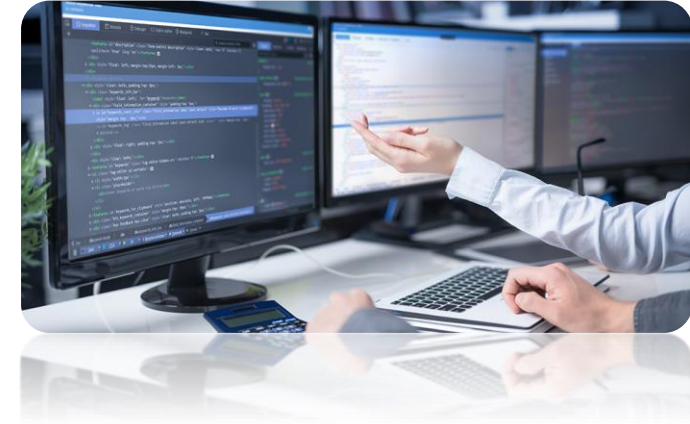
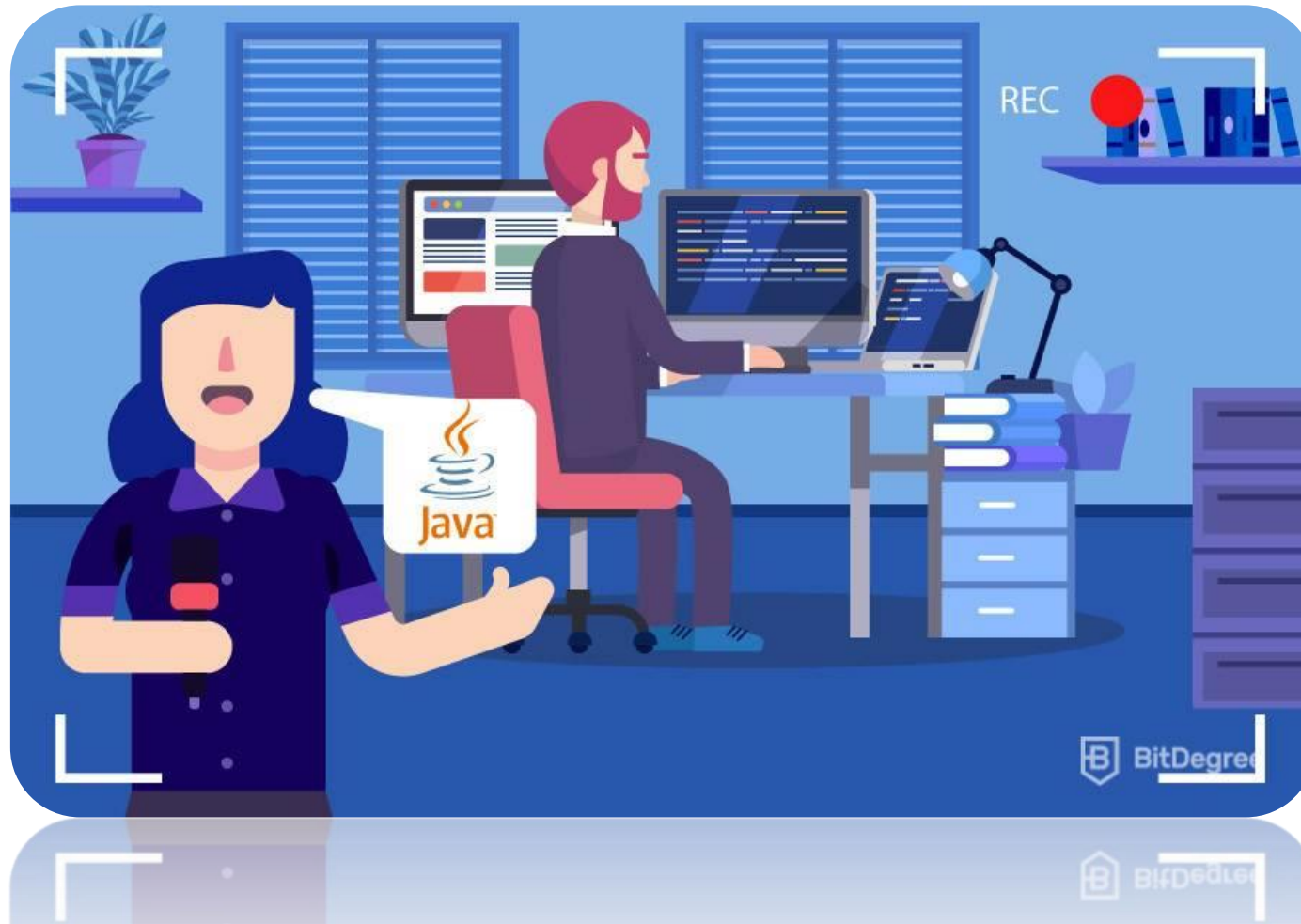
DÉPARTEMENT INFORMATIQUE, FACULTÉ DES SCIENCES

UNIVERSITÉ ABDELMALEK ESSAÂDI, TÉTOUAN

Avant-propos

MODULE : DÉVELOPPEMENT D'APPLICATIONS EN JAVA

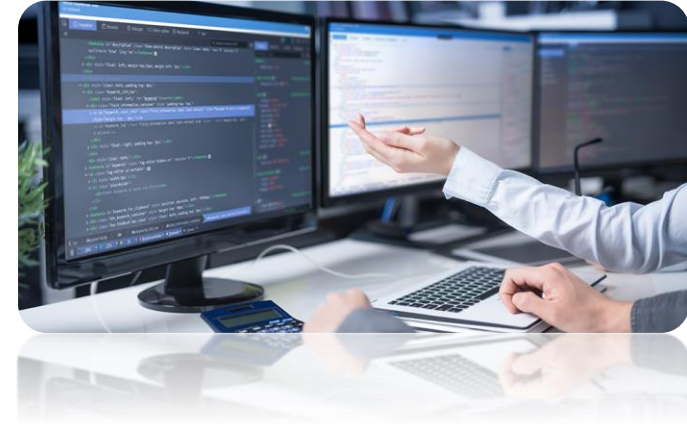
Avant-propos : Job !?



Avant-propos : Job !?

Devenir :

- ✓ Développeur en logiciel JAVA
- ✓ Développeur Java
- ✓ Développeur d'applications Java



Compétences techniques

- ✓ La maîtrise des technologies/frameworks de développement d'application Java : Java SE, Java FX, Java ME, Java EE,
- ✓ La manipulation avec de bases de données en Java (ex SGBD : MySQL, Oracle, SQLite, ...).

Avant-propos : Objectifs du module

1. Maîtriser les concepts fondamentaux de la programmation Java.
2. Concevoir et créer des interfaces utilisateur graphiques interactives avec AWT, Swing et JavaFX.
3. Comprendre et mettre en œuvre l'intégration de bases de données dans les applications Java.
4. Développer des compétences avancées en programmation orientée objet, en gestion des événements utilisateur et en manipulation de données.

Plan du cours :

Développement d'applications en Java

Élément 1 : Fondamentaux et POO en Java

- ✓ Introduction à Java et à l'environnement de développement.
- ✓ Fondamentaux de la programmation Java
- ✓ Programmation Orienté objet en Java
- ✓ Collections Java (List, Set, Map) et leurs applications
- ✓ Manipulation de fichiers
- ✓ Gestion des erreurs et des exceptions

Élément 3 : Application Java en Swing/AWT

- ✓ Concepts des interfaces graphiques (événements, écouteurs).
- ✓ Découverte de Swing et AWT : composants de base (boutons, menus, tableaux).
- ✓ Gestion des événements utilisateur.

Élément 2 : Intégrité de Base de données

- ✓ Connexion d'une application Java à une base de données (JDBC)
- ✓ Requêtes SQL en Java
- ✓ Transactions et sécurité (SQL Injection, PreparedStatement).

Chapitre 1 : Fondamentaux de Java

-  Historique de Java
-  Caractéristiques principales
-  Java vs C/C++
-  JDK, JRE, JVM
-  Outils de développement
-  Organisation d'un projet Java
-  Structure d'un programme Java
-  Variables, types et Opérateurs
-  Entrées / Sorties
-  Structures de Contrôle
-  Les tableaux

Chapitre 2 : POO en Java

-  Concepts de la POO
-  Classes et objets
-  Encapsulation
-  Héritage
-  Polymorphisme
-  Interfaces
-  Gestion des exceptions
-  Collections Java
-  Manipulation de fichiers

Élément 1 : Fondamentaux et POO en Java

Chapitre 1 : Fondamentaux de Java

-  Historique de Java
-  Caractéristiques principales
-  Java vs C/C++
-  JDK, JRE, JVM
-  Outils de développement
-  Organisation d'un projet Java
-  Structure d'un programme Java
-  Variables, types et Opérateurs
-  Entrées / Sorties
-  Structures de Contrôle
-  Les tableaux

Élément 1 : Fondamentaux et POO en Java



Historique de Java

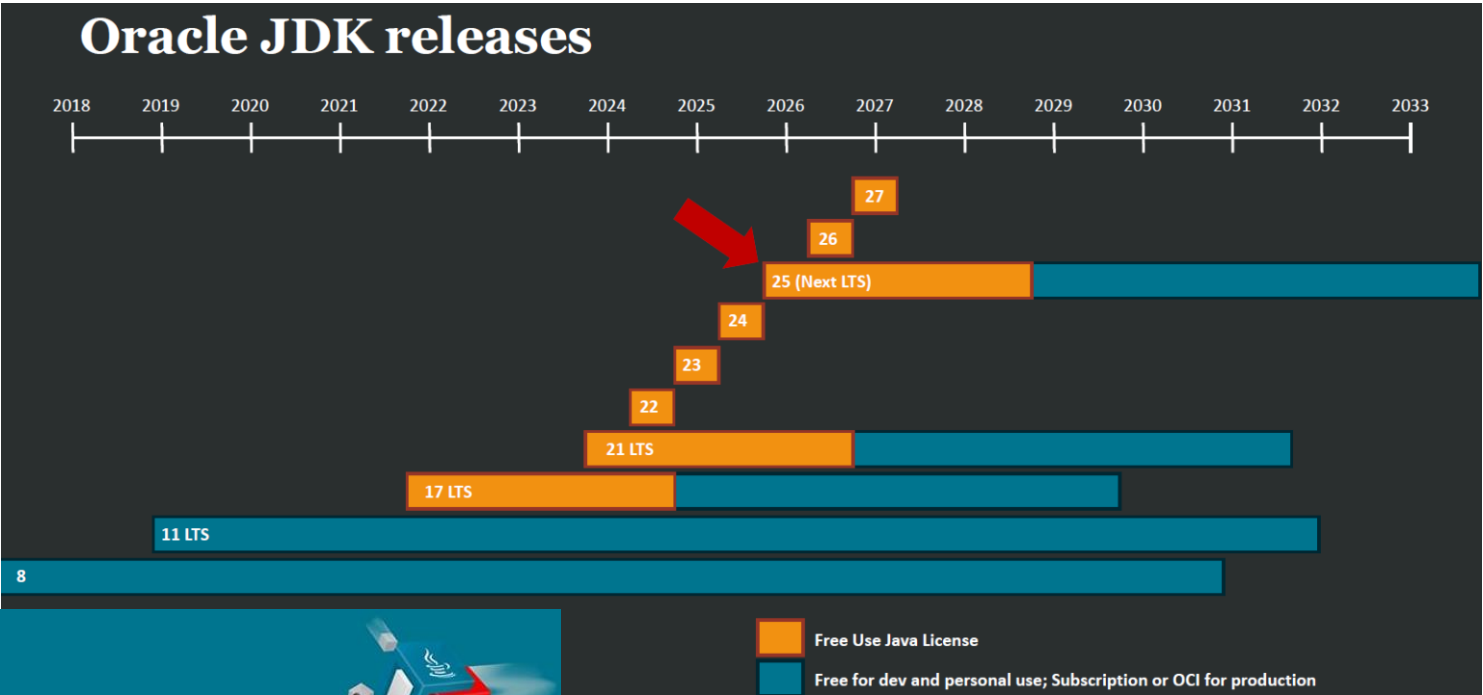
Logo de Sun Microsystems, créateur de Java



James Gosling, père fondateur de Java

- 1991** Début du projet "Green" chez Sun Microsystems par James Gosling
- 1995** Première version publique de Java (Java 1.0)
- 1996** Introduction de Java Applets pour le web
- 1998** Java 2 (J2SE 1.2) - Refonte majeure de la plateforme
- 2004** Java 5 - Ajout des génériques, annotations, autoboxing
- 2010** Oracle acquiert Sun Microsystems
- 2014** Java 8 - Expressions lambda, Stream API
- 2018** Nouveau cycle de release tous les 6 mois
- 2023** Java 21 (LTS) - Dernière version stable
- 2025** Java 25 now available

September 16, 2025



Java 25 now available

Learn more about the latest release



Caractéristiques principales de Java

- ✓ Java est un langage de programmation orienté objet, polyvalent, et indépendant de la plateforme.
- ✓ Créé par Sun Microsystems en 1995, maintenant sous Oracle.
- ✓ Utilisé dans le développement web, applications mobiles (Android), logiciels d'entreprise.
- ✓ Philosophie de Java : 'Écrire une fois, exécuter partout' **Write Once, Run Anywhere WORA**).



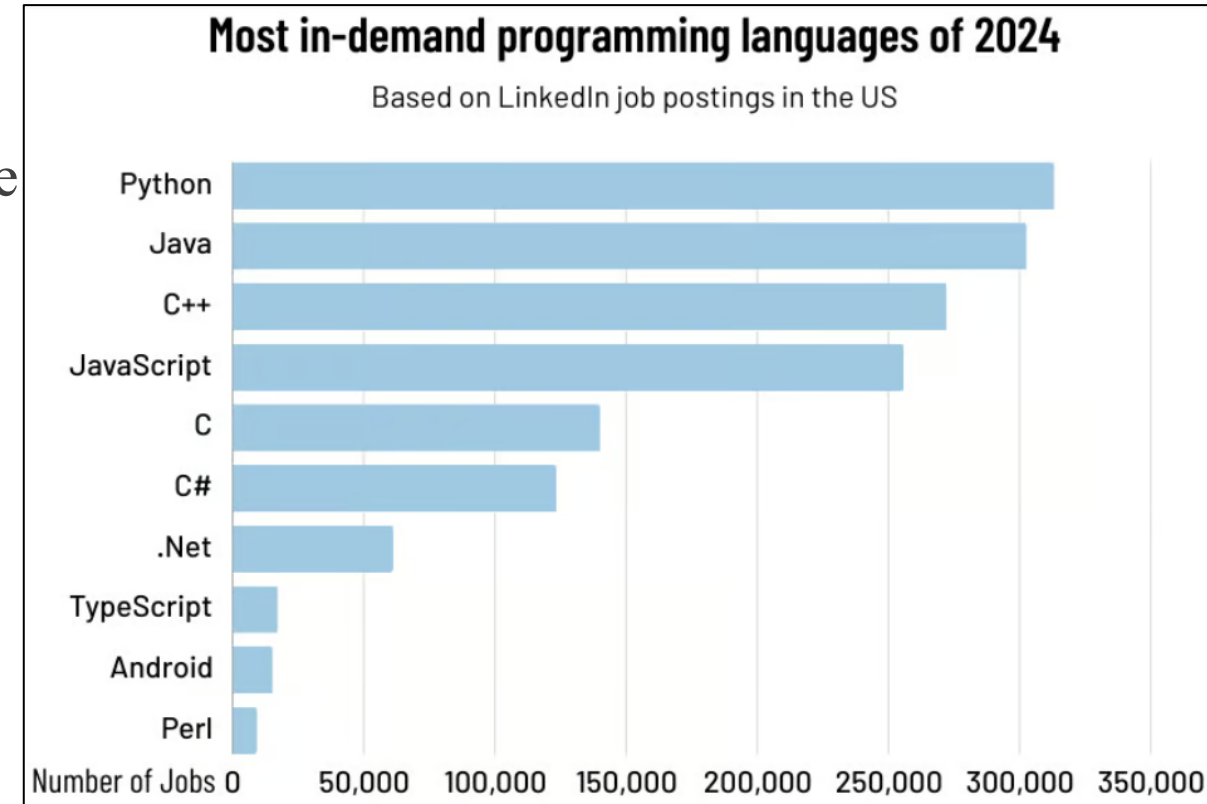
Introduction
to
Java
Programming

1989

١٩٨٩

Caractéristiques principales de Java

- ✓ **Robustesse** : Gestion automatique de la mémoire, sécurité renforcée.
- ✓ **Portabilité** : Fonctionne sur n'importe quel système d'exploitation avec la JVM.
- ✓ **Polyvalence** : Développement d'applications web, mobiles, de bureau.
- ✓ **Communauté** : Large communauté et nombreux frameworks.



Caractéristiques principales de Java

Orienté objet

Tout en Java est un objet, facilitant la modularité et la réutilisation du code

Portabilité (WORA)

Write Once, Run Anywhere - Le bytecode Java s'exécute sur n'importe quelle JVM

Garbage Collector

Gestion automatique de la mémoire, libérant le développeur de cette tâche

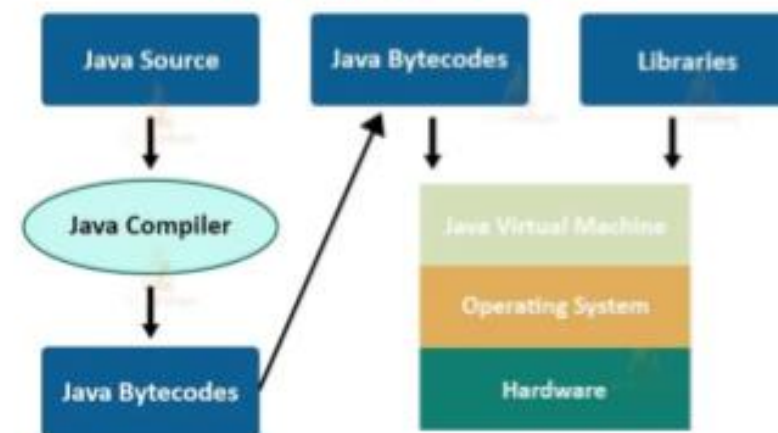
Sécurité intégrée

Architecture de sécurité robuste avec sandbox d'exécution et vérification du bytecode

Bibliothèque standard riche

API complète couvrant réseau, interface graphique, collections, concurrence, etc.

Working of JVM



Architecture de la JVM (Java Virtual Machine) permettant la portabilité du code Java

Caractéristiques principales de Java

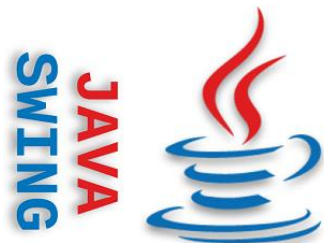
Just a moment...



Java mobile



J2ME platform



Caractéristiques principales de Java: Java SE

- ✓ Le terme « **Java** » fait bien évidemment référence à un langage.
- ✓ Mais également à une plate-forme : son nom complet est « **Java SE** » pour **Java Standard Edition**, et était anciennement raccourci « **J2SE** ».
- ✓ **Java SE** est constituée de nombreuses bibliothèques, ou **API**.
- ✓ Par exemple : *java.lang*, *java.io*, *java.math*, *java.util*, etc.
- ✓ **Java SE** regroupe toutes les bibliothèques qui contiennent un nombre important de classes et de méthodes prêtes à l'emploi pour effectuer toutes sortes de tâches et afin de développer les applications Desktop.



Java EE (J2EE ou JEE)

- ✓ **Java EE** signifie **Java Enterprise Edition**.
- ✓ **Java EE** est à une extension de la plate-forme standard.
- ✓ La plate-forme **Java EE** est construite sur le langage Java et la plate-forme Java SE.
- ✓ L'objectif majeur de **Java EE** est de faciliter le développement d'applications web robustes et distribuées, déployées et exécutées sur un **serveur d'applications**.
- ✓ **Java EE 8** était la dernière version de **Java EE** d'Oracle, et maintenant, Oracle a déplacé **Java EE** vers une fondation open source (*Eclipse Foundation*), et elle s'appelle **Jakarta EE**.



Java vs C/C++

Caractéristique	Java	C/C++
Gestion mémoire	Automatique (Garbage Collector)	Manuelle (malloc/free, new/delete)
Pointeurs	Références uniquement	Pointeurs explicites
Héritage	Simple uniquement	Simple et multiple
Portabilité	Élevée (WORA)	Dépendante de la plateforme
Compilation	Bytecode → JVM	Code machine natif

Top 8 in-demand programming languages

1

Python



2

Java



3

JavaScript



4

TypeScript



5

PHP



6

C++



7

Swift



8

Kotlin





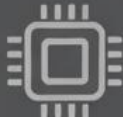
JAVA

- Mémoire Auto (GC)
- Portable
- OOP Seulement
- Aplis Web/Mobile
- Easier Facile



C++

- Manual Memory (Ponters)
- Platform Dependent
- Multi-Paridagm
- Games/OS/Embedded
- Complex Syntax



Faster Dev



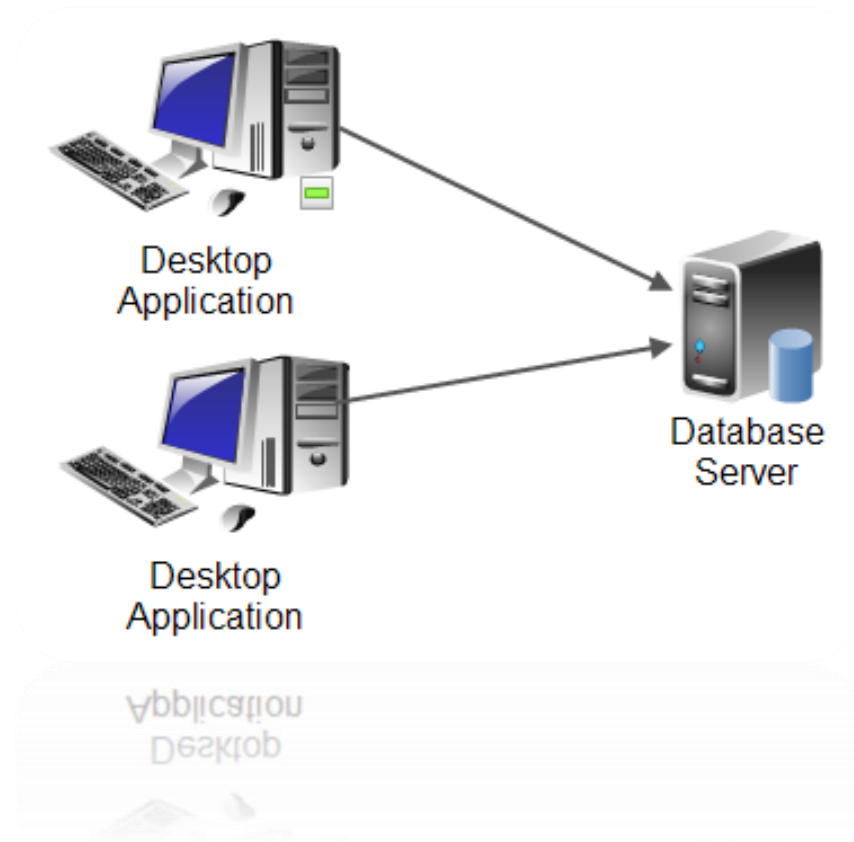
Desktop || Web Application

Une application de bureau est installée, lancée et exécutée localement.



Une application Web fonctionne via un navigateur, l'utilisant comme un environnement d'exécution.

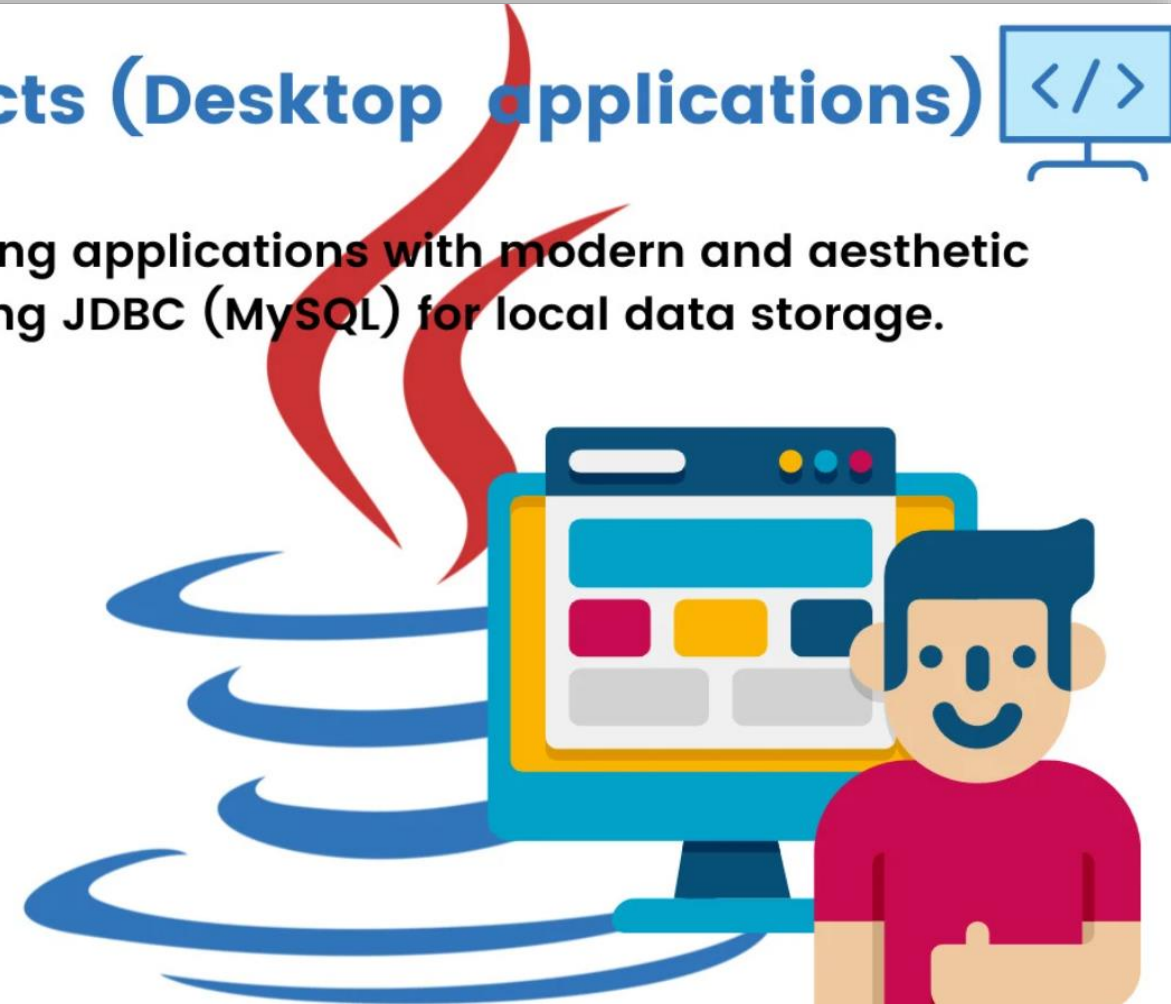
Desktop Application



Java Projects (Desktop applications)

- Building Swing applications with modern and aesthetic designs ,using JDBC (MySQL) for local data storage.

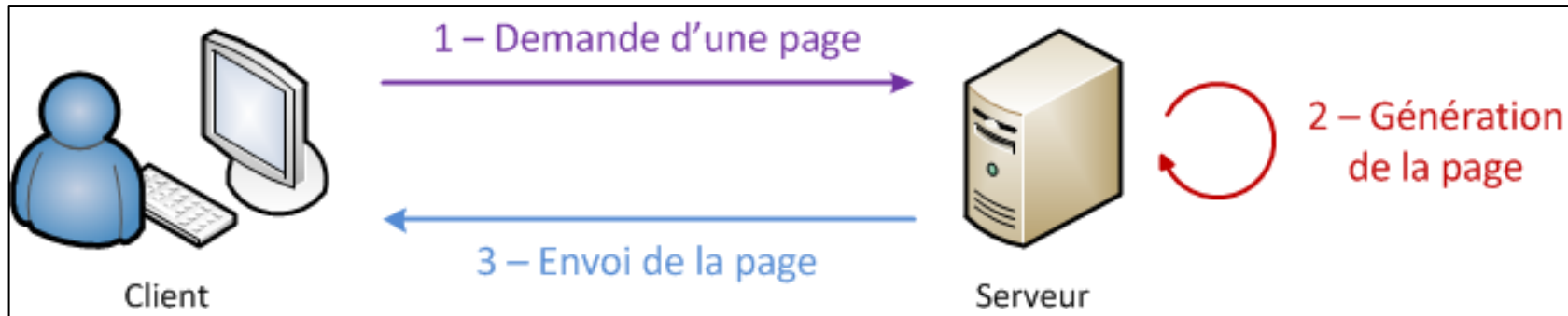
MySQL®
SWING



Src : <https://www.fiverr.com/driss20/make-a-stylish-desktop-application-using-java-and-mysql>

Application Web

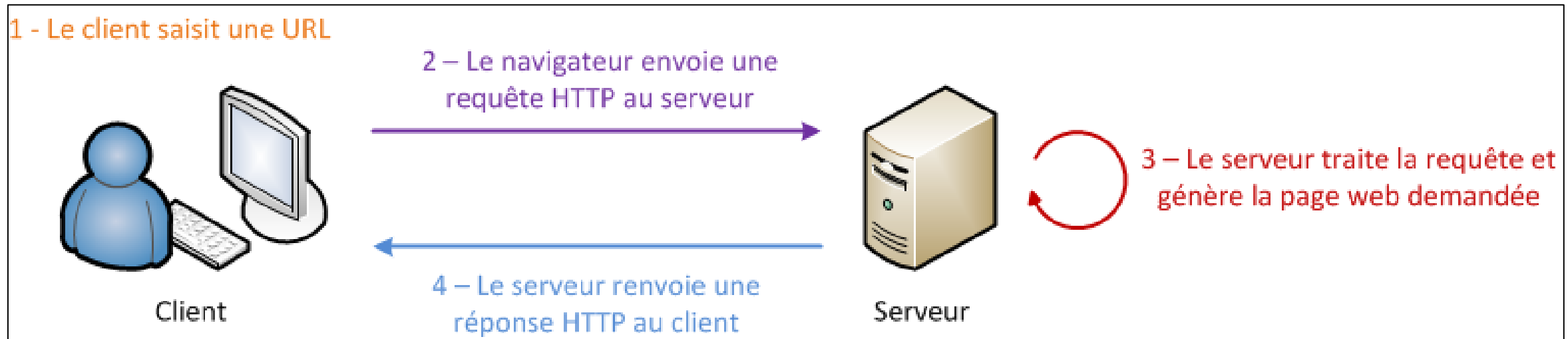
- ✓ La communication qui s'effectue entre le client et le serveur est régie par des règles bien définies : le protocole HTTP.



1. L'utilisateur saisit une URL dans la barre d'adresses de son navigateur ;
2. Le navigateur envoie alors une requête HTTP au serveur pour lui demander la page correspondante ;
3. Le serveur reçoit cette requête, l'interprète et génère alors une page web qu'il va renvoyer au client par le biais d'une réponse HTTP ;
4. Le navigateur reçoit, via cette réponse, la page web finale, qu'il affiche alors à l'utilisateur.

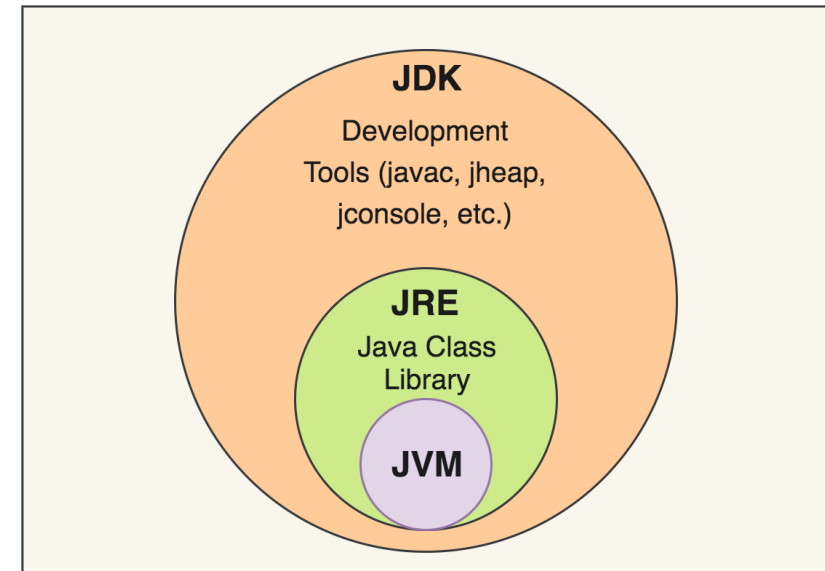
Application Web

- ✓ Les données sont échangées entre le client et le serveur via le protocole HTTP ;
- ✓ Le client ne comprend que les langages de présentation de l'information, en d'autres termes les technologies HTML, CSS et Javascript ;
- ✓ Les pages sont générées sur le serveur de manière dynamique, à partir du code source du site.



L'environnement de développement Java (IDE)

- ✓ JDK (Java Development Kit) : Fournit outils pour compiler, déboguer, exécuter le code Java.
- ✓ JRE (Java Runtime Environment) : Environnement nécessaire pour exécuter les applications Java.
- ✓ JVM (Java Virtual Machine) : Simule l'exécution des programmes Java.

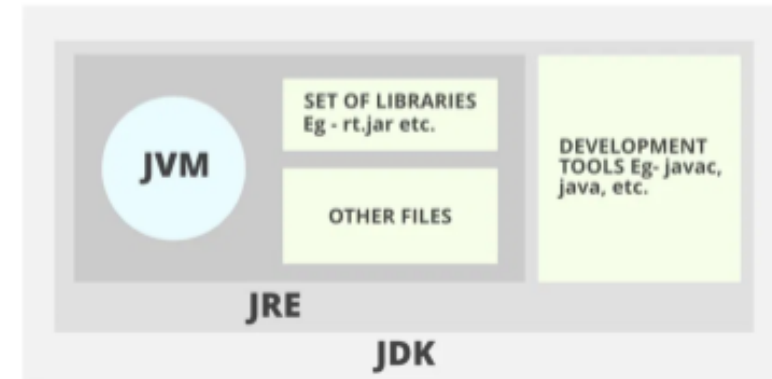


L'environnement de développement Java (IDE)

JDK (Java Development Kit)

Environnement complet pour le développement d'applications Java

- Compilateur Java (javac)
- Outils de développement (javadoc, jar, etc.)
- Débogueur (jdb)
- Inclut le JRE



Relation entre JDK, JRE et JVM - Structure en couches

JRE (Java Runtime Environment)

Environnement d'exécution pour les applications Java

- Bibliothèques de classes Java (API)
- Chargeur de classes
- Inclut la JVM

JVM (Java Virtual Machine)

Machine virtuelle qui exécute le bytecode Java

- Interprète le bytecode
- Compilateur JIT (Just-In-Time)
- Garbage Collector
- Gestionnaire de mémoire

L'environnement de développement Java (IDE)

Outils de développement Java

Ligne de commande

> javac

Compilateur Java qui transforme le code source (.java) en bytecode (.class)

▶ java

Lanceur de la JVM qui exécute le bytecode Java

📦 jar

Outil pour créer et gérer des archives JAR (Java ARchive)

🔍 jdb

Débogueur Java en ligne de commande

Outils de build

⚙️ Maven

Gestion de dépendances et automatisation de build

☰ Gradle

Système de build flexible basé sur Groovy/Kotlin

Environnements de Développement Intégrés (IDE)



Eclipse IDE

Open source, extensible via plugins



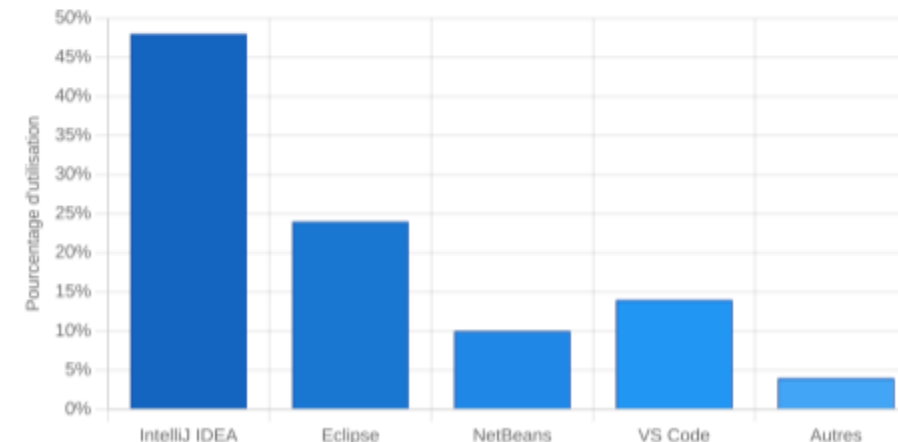
Apache NetBeans

Modulaire, multi-langage



IntelliJ IDEA

Commercial, très puissant



Java Downloads | Oracle

+

oracle.com/java/technologies/downloads/

🔍

🇺🇸

👤

View Accounts

🗨️

Contact Sales

Products Industries Resources Customers Partners Developers Company

Java > Technical Details >

Java Downloads



Tools and resources

Java downloads

Java archive



Looking for other Java downloads?

OpenJDK Early Access Builds

JRE for Consumers

Java 23, Java 21, and earlier versions available now

JDK 23 is the latest release of the Java SE Platform.

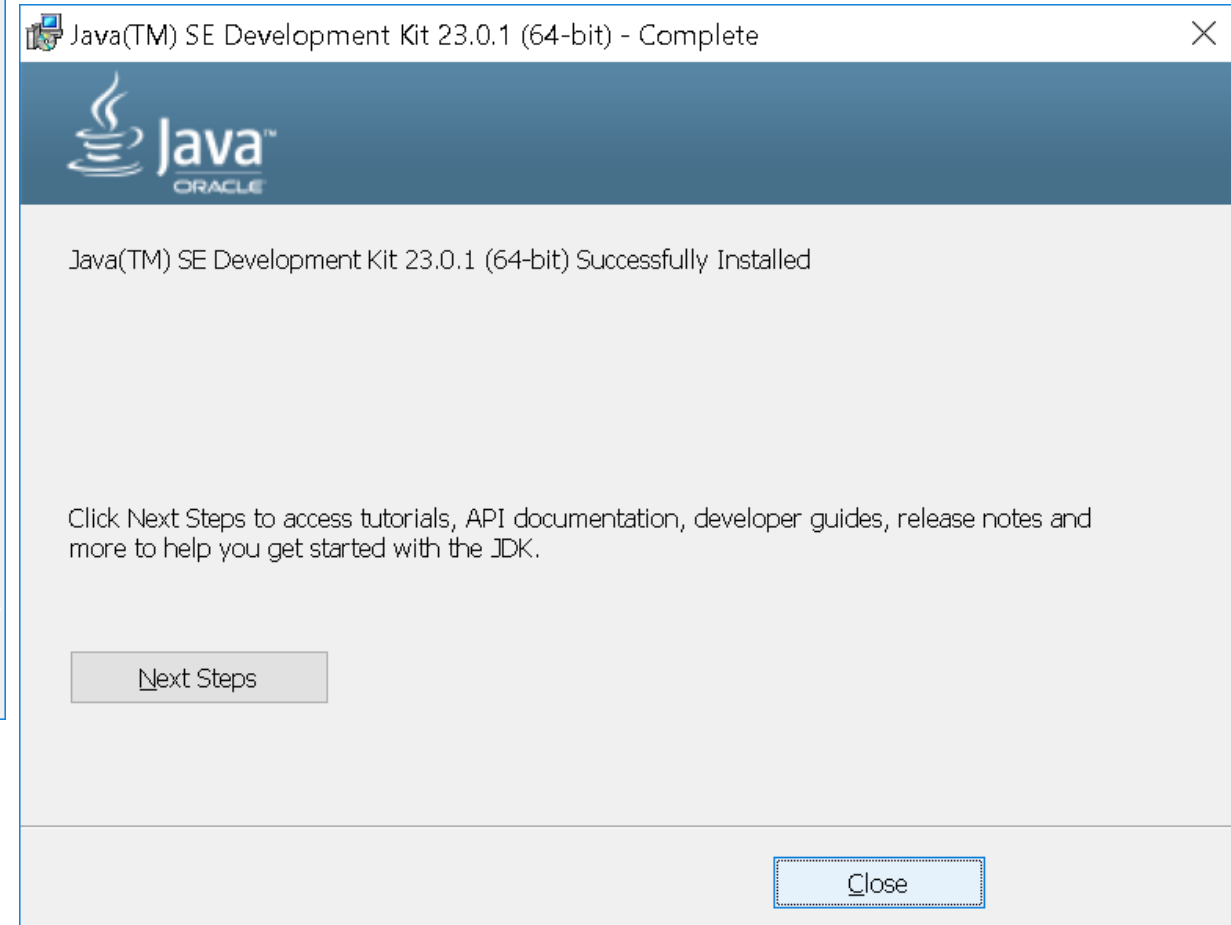
JDK 21 is the latest *Long-Term Support (LTS)* release of the Java SE Platform.

Learn about Java SE Subscription

Pr. Otman ABDOUN - Département Informatique

LICENCE IIDL / S5 : DÉVELOPPEMENT D'APPLICATIONS EN JAVA

24



C:\WINDOWS\system32\cmd.exe

Microsoft Windows [version 10.0.16299.1127]
(c) 2017 Microsoft Corporation. Tous droits réservés.

C:\Users\User>java --version

java 23.0.1 2024-10-15

Java(TM) SE Runtime Environment (build 23.0.1+11-39)

Java HotSpot(TM) 64-Bit Server VM (build 23.0.1+11-39, mixed mode, sharing)

C:\Users\User>_

L'environnement de développement Java (IDE)

✓ **IDE** - Integrated Development Environment - Environnement de développement intégré; un IDE n'est pas un éditeur de texte.

✓ **IDE** a un ensemble de fonctionnalités :

- La vérification de la syntaxe du langage ;
- La détection des erreurs ;
- L'auto-complétion ;
- Un compilateur ;
- Un serveur.

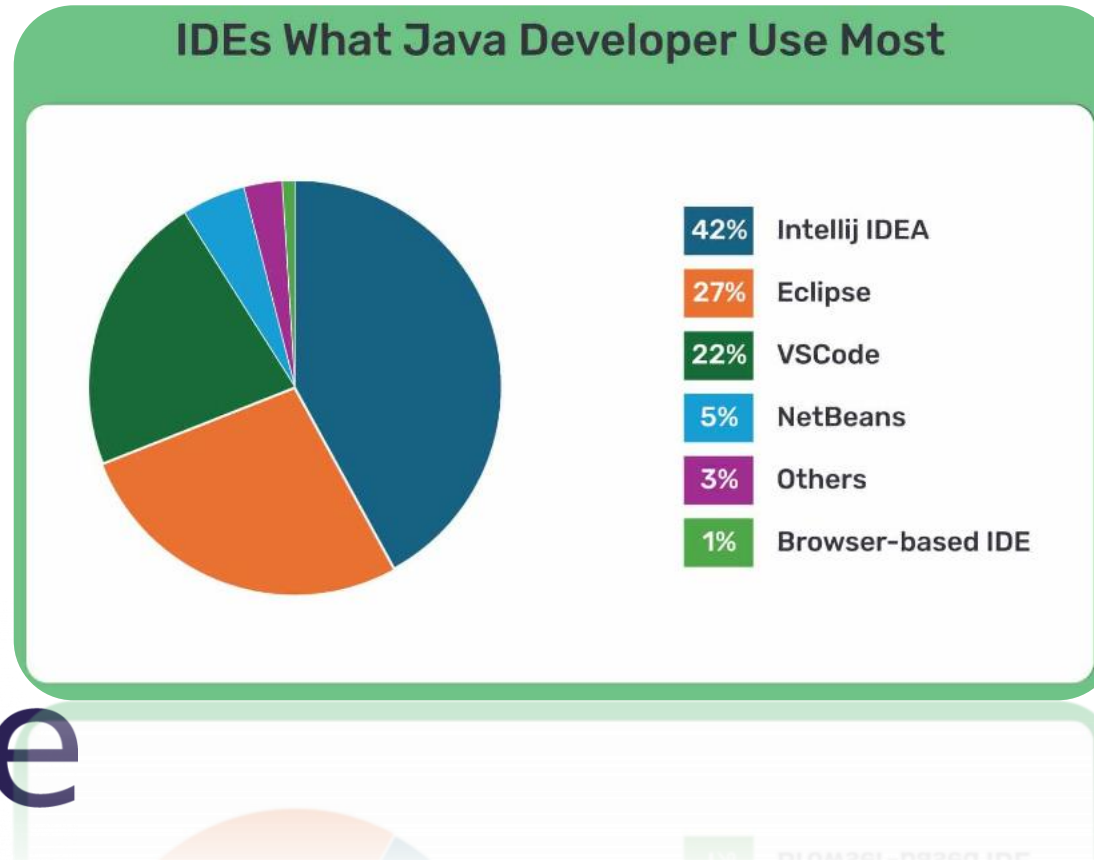
✓ Principaux **IDEs** : Eclipse, IntelliJ IDEA, NetBeans.



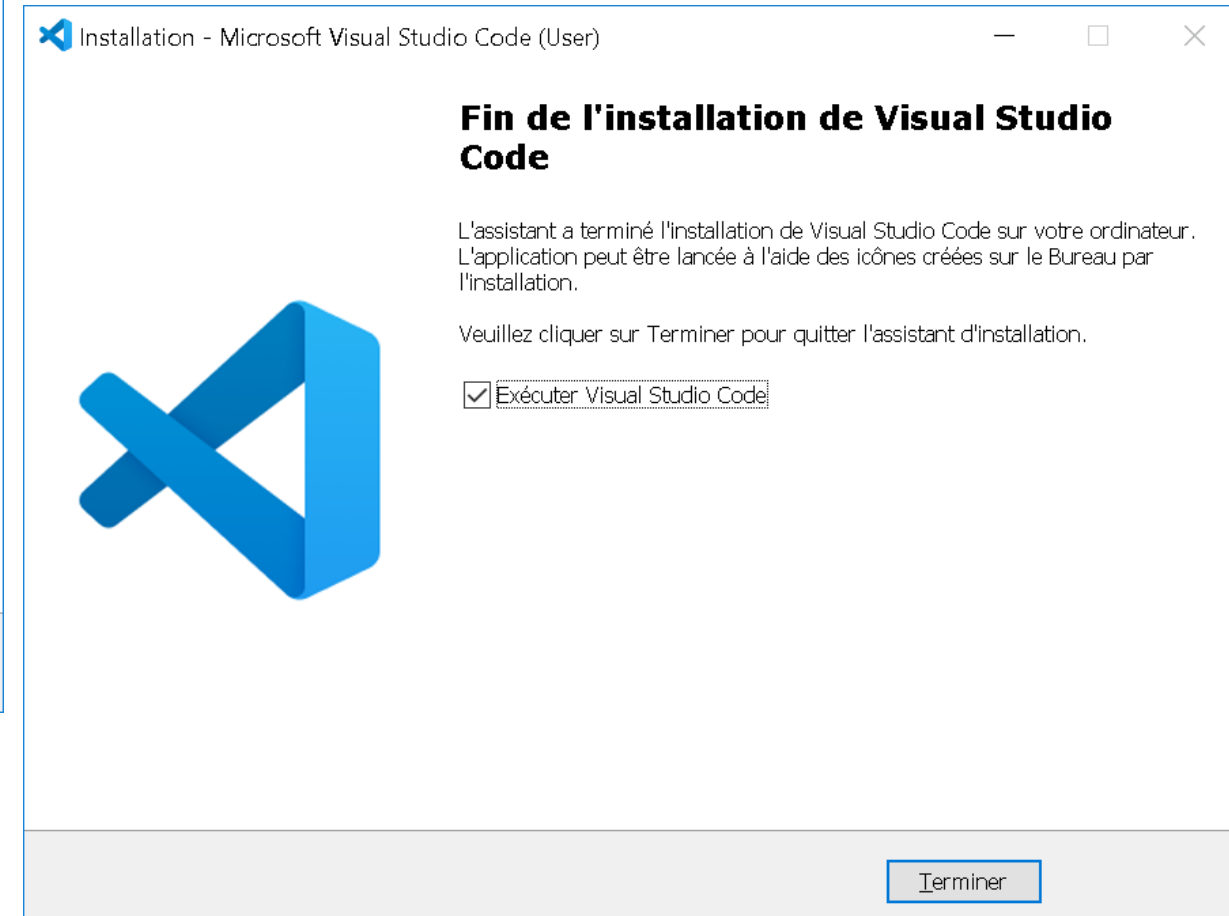
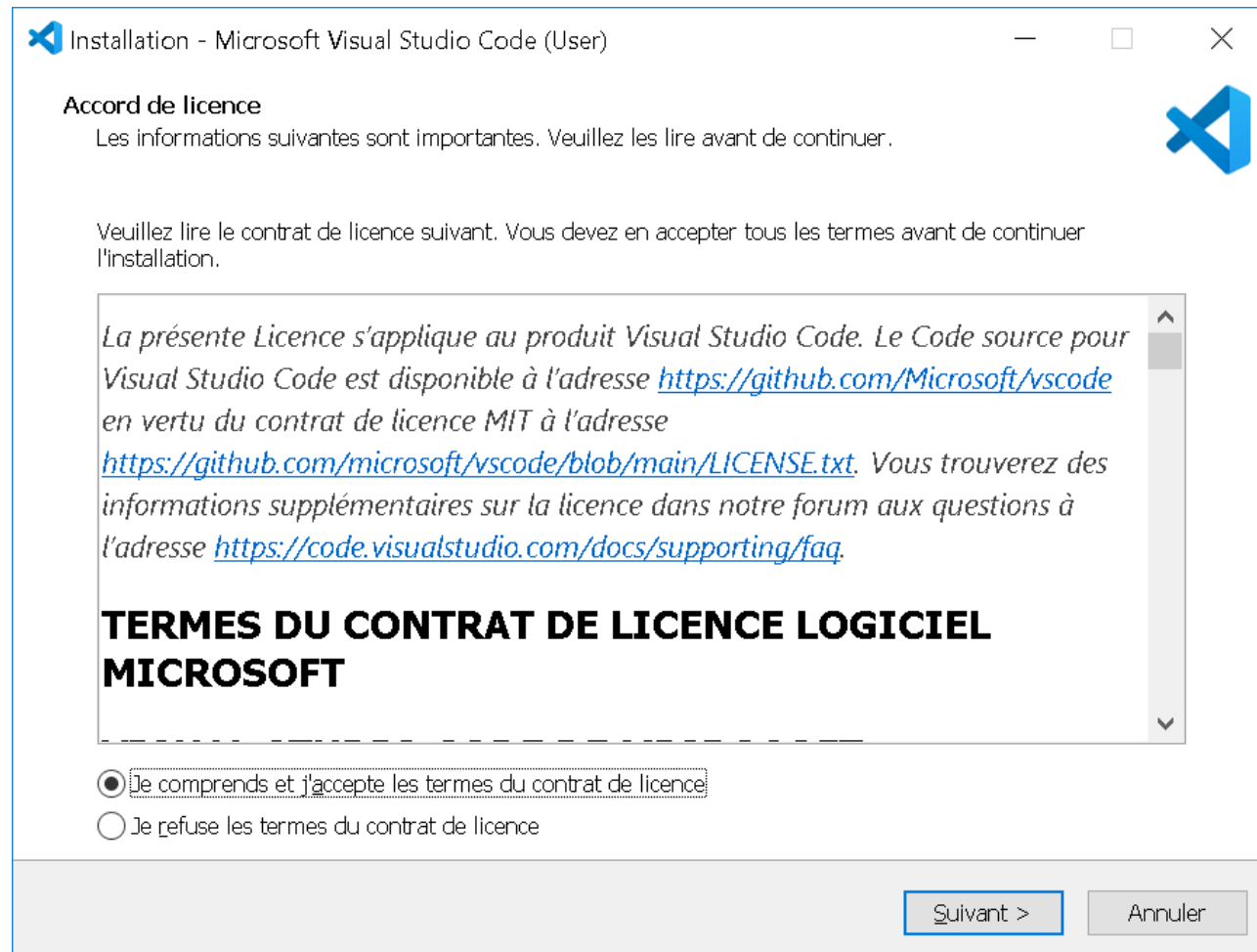
Pratique > IDE for Java



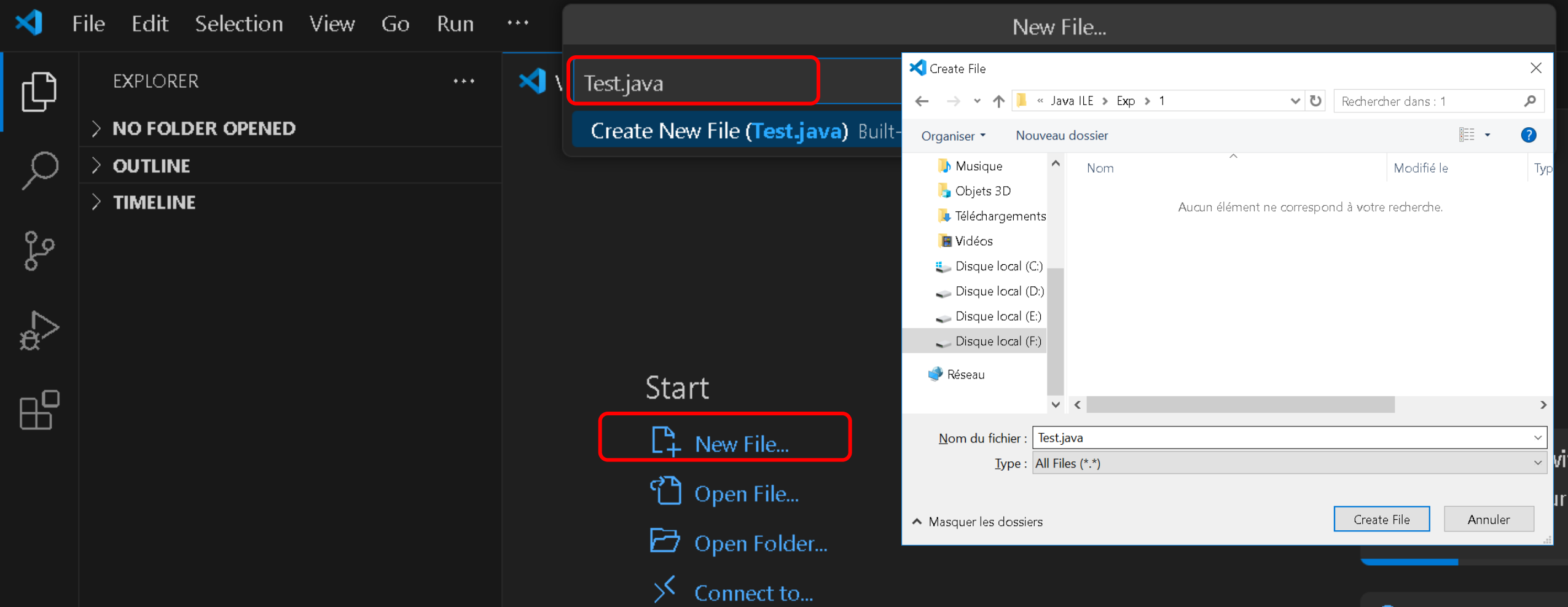
Apache
NetBeans IDE



Visual Studio Code



Get started with Visual Studio Code - VSC !



FileEditSelectionViewGoRun

Search

EXTENSIONS: MARKETPLACE

@id:vscjava.vscode-java-pac

Extension Pack f... 158ms

Popular extensions for Java ...

Microsoft

Extension: Extension Pack for Java x

Test.java 1

Extension Pack for Java v0.29.0

Microsoft microsoft.com 31,249,791 ★★★★★ (80)

Popular extensions for Java development that provides Java IntelliSense, debugging, testing, ...

Disable Uninstall Switch to Pre-Release Version Auto Update

DETAILS FEATURES CHANGELOG

Extension Pack (7)

Language Support for Java(TM) ... 3992ms

Java Linting, Intellisense, formatting, refactor...

Red Hat

Debugger for Java 96ms

A lightweight Java debugger for Visual Stud...

Microsoft

Test Runner for Java 81ms

Run and debug JUnit or TestNG test cases.

Microsoft

Categories

Programming Languages

Linters Debuggers

Formatters Snippets

Extension Packs

Resources

Marketplace

Help Red Hat improve its extensions by allowing them to collect usage data. Read our privacy statement and learn how to opt out.

Source: Language Support for Java(TM) by Red Hat

Accept Deny

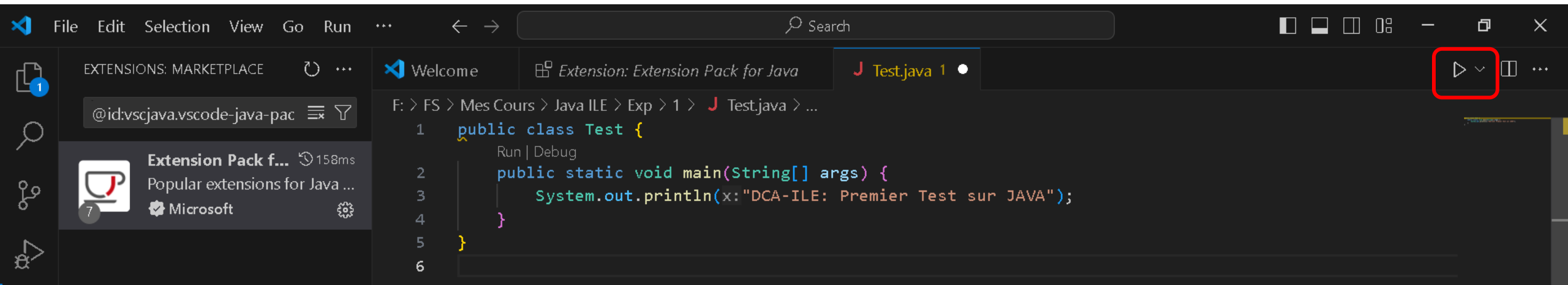
Java: Searching... - 0%

Pr. Otman ABDOUN - Département Informatique

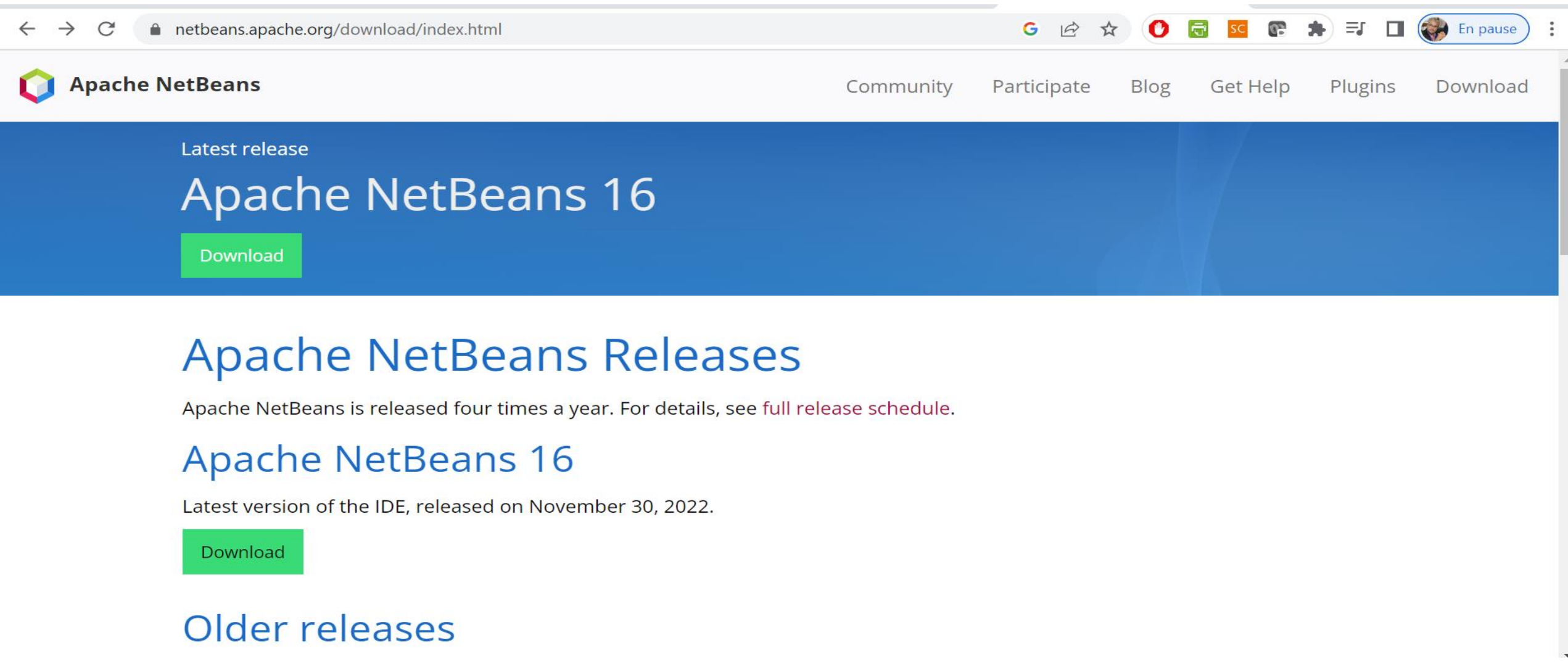
LICENCE IIDL / S5 : DÉVELOPPEMENT D'APPLICATIONS EN JAVA

31

Premier test sur VSC



Pratique > IDE for Java



The screenshot shows the Apache NetBeans website. The browser's address bar displays 'netbeans.apache.org/download/index.html'. The website's navigation bar includes links for 'Community', 'Participate', 'Blog', 'Get Help', 'Plugins', and 'Download'. The main content area features a blue header with the text 'Latest release' and 'Apache NetBeans 16', accompanied by a green 'Download' button. Below this, the section 'Apache NetBeans Releases' is introduced, stating that the IDE is released four times a year and providing a link to the 'full release schedule'. The 'Apache NetBeans 16' section follows, noting the latest version was released on November 30, 2022, and also includes a green 'Download' button. The page concludes with a link to 'Older releases'.

← → ↻ netbeans.apache.org/download/index.html

Apache NetBeans

Community Participate Blog Get Help Plugins Download

Latest release

Apache NetBeans 16

Download

Apache NetBeans Releases

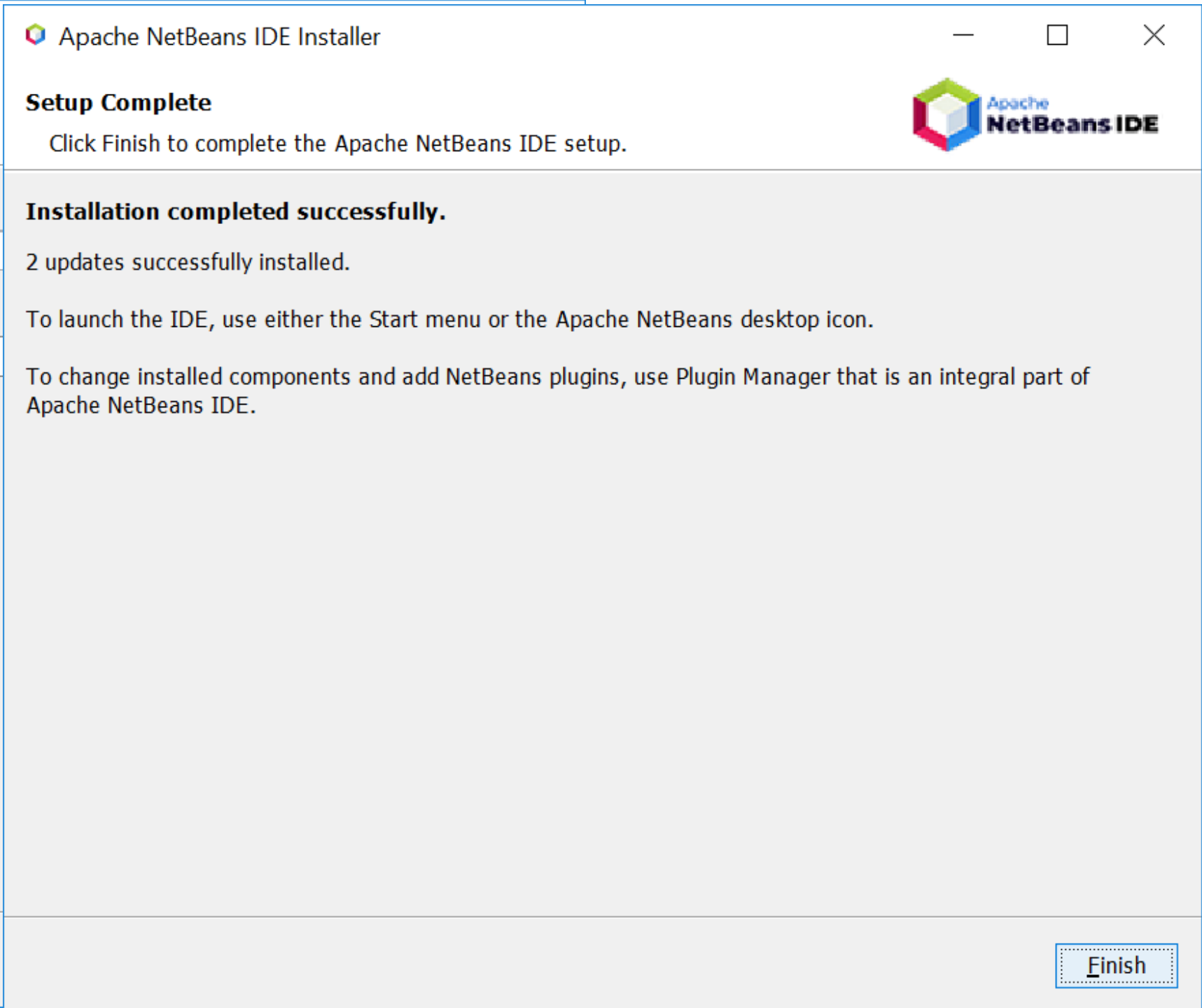
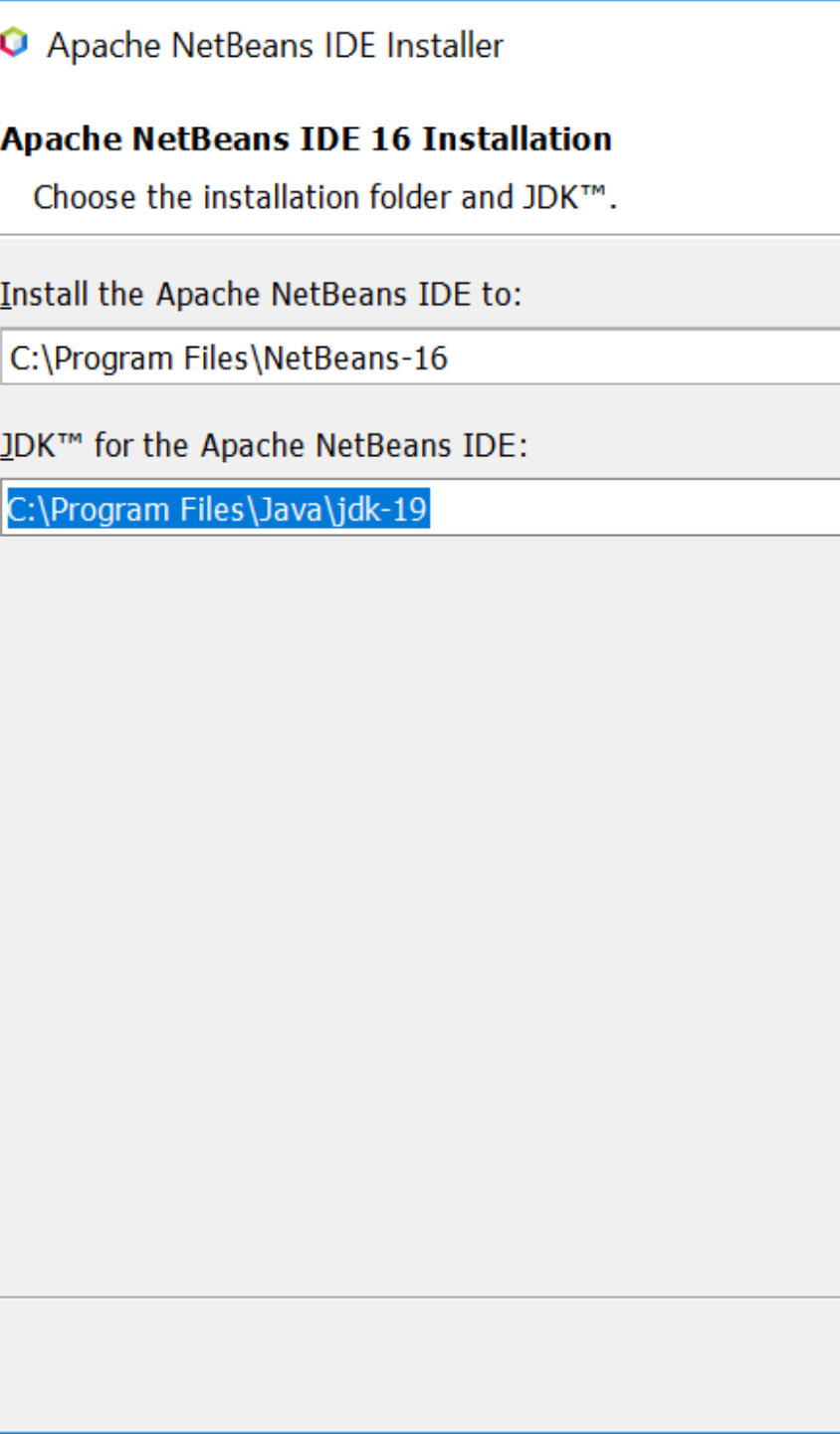
Apache NetBeans is released four times a year. For details, see [full release schedule](#).

Apache NetBeans 16

Latest version of the IDE, released on November 30, 2022.

Download

Older releases



Organisation d'un projet Java

1 Fichiers source (.java)

Contiennent le code source Java organisé en classes

```
// HelloWorld.java
public class HelloWorld {
// code source
}
```

2 Compilation (javac)

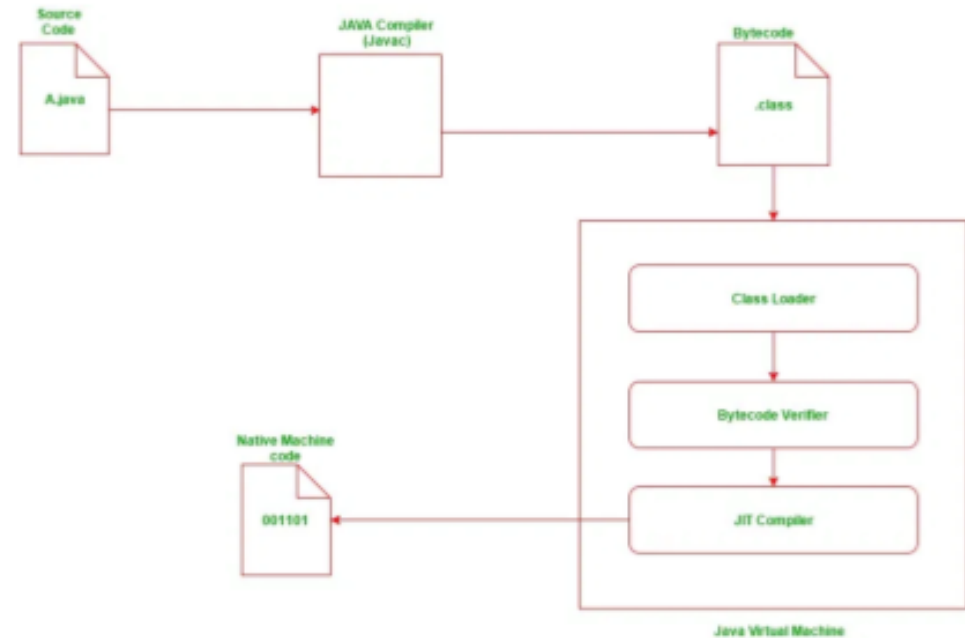
Transformation du code source en bytecode

```
$ javac HelloWorld.java
// Génère HelloWorld.class
```

3 Exécution (java)

Interprétation du bytecode par la JVM

```
$ java HelloWorld
```



4 Organisation par packages

Regroupement logique des classes dans des packages

```
package com.example.projet;

public class MaClasse {
//
code
}
```

Exemple minimal d'un programme Java

HelloWorld.java

```
1 // Un programme Java minimal
2 public class HelloWorld {
3 // La méthode principale
4 public static void main(String[] args) {
5     System.out.println( "Bonjour, Java !" );
6 }
7 }
```

Compilation et exécution

```
1 $ javac HelloWorld.java
2 $ java HelloWorld
3 Bonjour, Java !
```

Classe publique

Le nom du fichier doit correspondre au nom de la classe publique

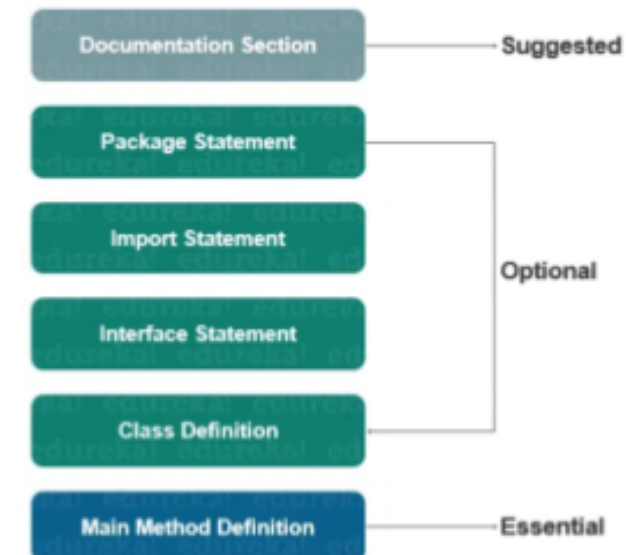
Méthode main

Point d'entrée obligatoire:

public static void main(String[] args)

Sortie standard

System.out.println() affiche du texte dans la console



Structure d'un Programme Java

Le code de la classe doit être enregistré dans un fichier de même nom (casse comprise) que la classe

HelloWorld.java

Le point d'entrée pour l'exécution est la méthode `main()`

① Tout code java doit être défini à l'intérieur d'une classe

```
public class HelloWorld {
```

```
    public static void main(String[] args) {  
        int i = 0;  
  
        while (i < 5) {  
            System.out.println("Hello World !");  
            i++;  
        }  
    }  
}
```

② La description de la classe est effectuée à l'intérieur d'un bloc { }

Compilation :
`javac HelloWorld.java`



HelloWorld.java

javac



HelloWorld.class

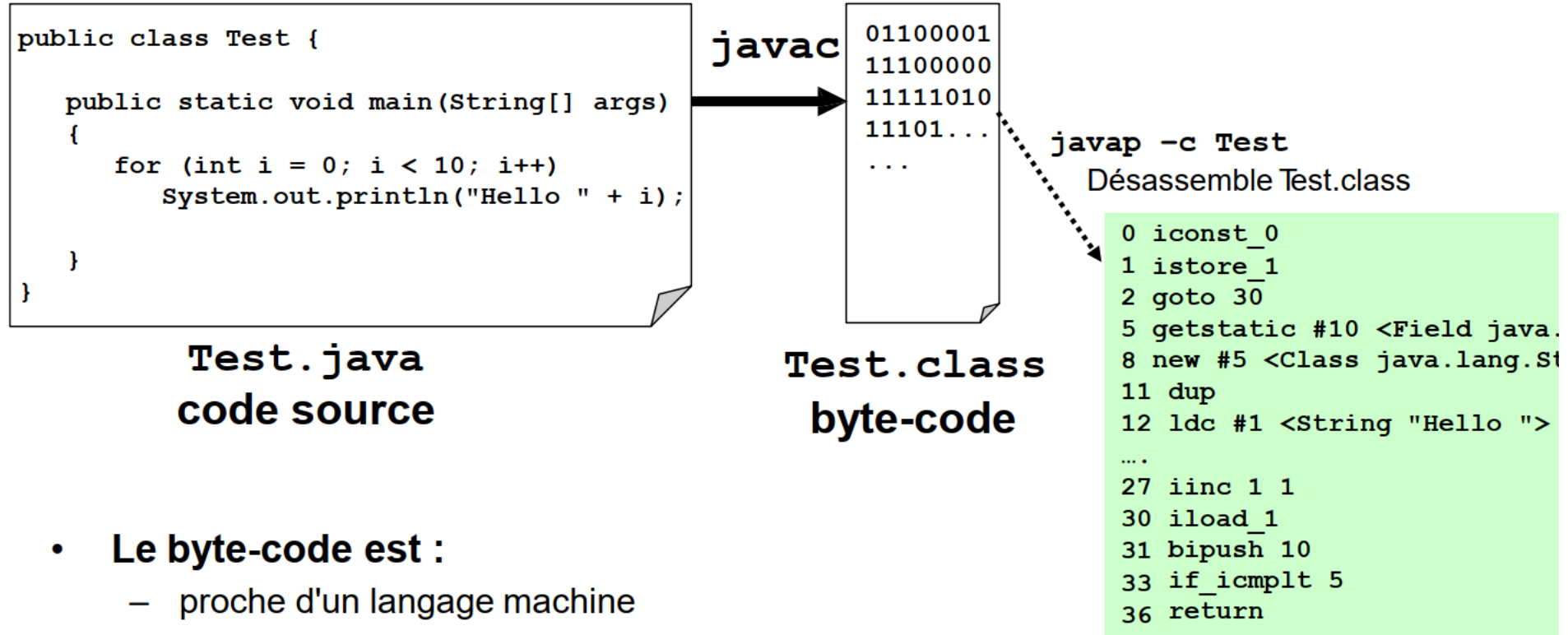
Exécution :
`java HelloWorld`

java

```
Hello World !  
Hello World !  
Hello World !  
Hello World !  
Hello World !
```

Structure d'un Programme Java

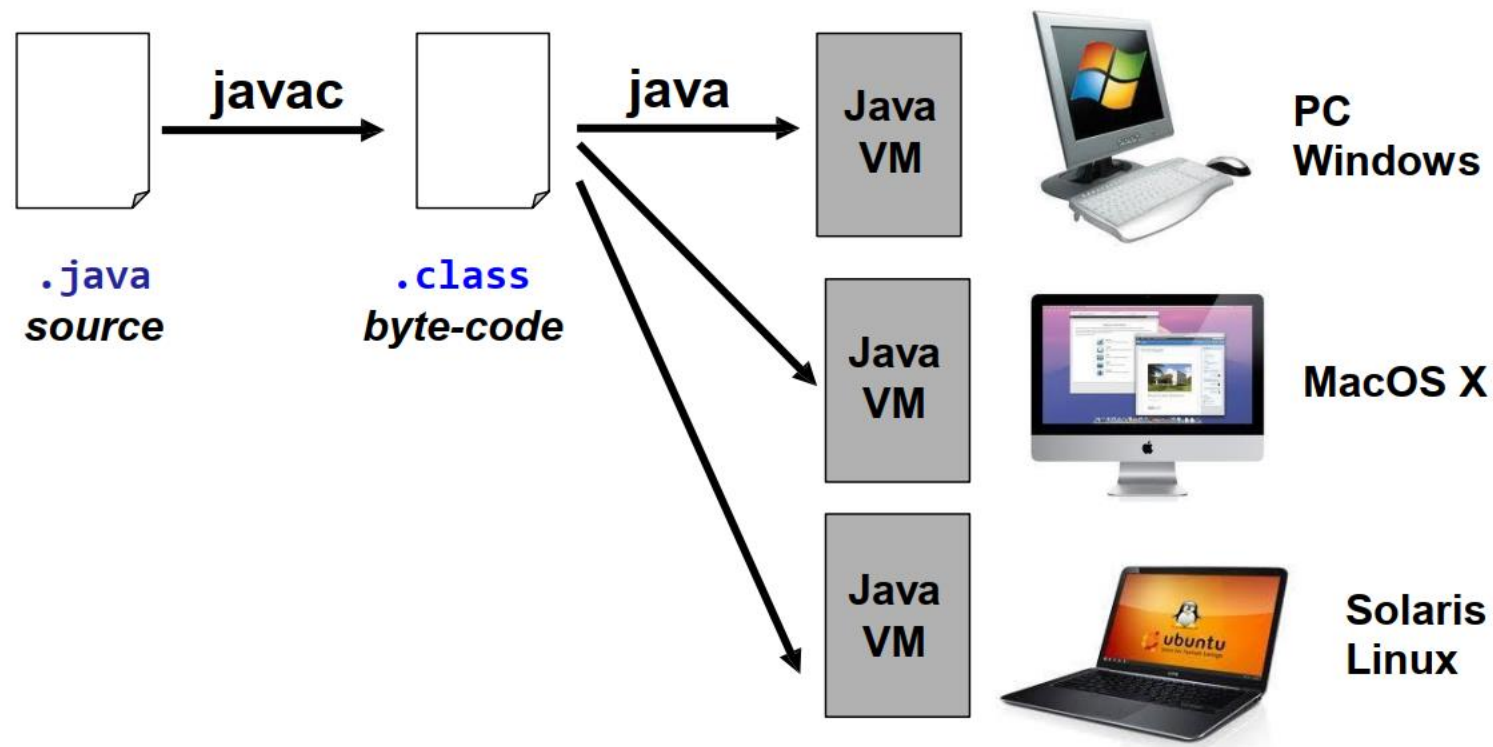
- **Compilation d'un programme JAVA : génération de byte-code**



- **Le byte-code est :**
 - proche d'un langage machine
 - indépendant de la plateforme d'exécution (matériel + OS)

Structure d'un Programme Java

- **byte-code assure la portabilité des programmes Java**
 - langage d'une Machine Virtuelle
 - à l'exécution un interpréteur simule cette machine virtuelle



Structure d'un Programme Java

Structure de base :

```
public class NomDeLaClasse {  
    public static void main(String[] args) {  
        // code ici  
    }  
}
```

- **Explication des éléments : Classe** : Conteneur de code.
- **Méthode main** : Point d'entrée de l'application.
- **System.out.println** : Affiche du texte.

Exemple : Mon premier programme en Java, affiche 'Hello, World!'

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Entrées / Sorties en Java

Afficher un message sur l'écran :

- ✓ Pour afficher un message sur l'écran on utilise la méthode **print** ou **println** de la classe **System**.
- ✓ **Println** permet d'afficher le message sur console et retourner à la ligne.

La syntaxe :

```
System.out.println("Veuillez saisir un numero: ");
```

Sorties standard

- **System.out.println()**
Affiche du texte avec un retour à la ligne
- **System.out.print()**
Affiche du texte sans retour à la ligne
- **System.out.printf()**
Affiche du texte formaté (similaire à printf en C)

```
// Exemples de sorties  
System.out.println( "Bonjour" );  
System.out.print( "Sans retour à la ligne " );  
System.out.printf( "Nombre: %d, Texte: %s" , 42, "Java" );
```

Entrées / Sorties en Java

La classe Scanner :

- ✓ La classe Scanner comporte des méthodes consistent à lire un flux de données caractères par caractères ou bien ligne par ligne saisi par périphérique d'entrée tel que le clavier.

Exemple : La lecture d'un entier saisi par le clavier

```
Scanner sc=new Scanner(System.in);  
int numero=sc.nextInt();
```

Les classes **Scanner** et **System** sont des classes du package

java.util

Méthodes principales de Scanner

Méthode	Type lu	Description
nextLine()	String	Lit une ligne entière
next()	String	Lit un mot
nextInt()	int	Lit un entier
nextDouble()	double	Lit un nombre à virgule
nextBoolean()	boolean	Lit un booléen

Entrées / Sorties en Java

Entrées avec Scanner

```
import java.util.Scanner;

public class ExempleEntree {
    public static void main(String[] args) {
        // Création d'un objet Scanner
        Scanner sc = new Scanner(System.in);

        // Lecture de différents types
        System.out.print( "Entrez votre nom: " );
        String nom = sc.nextLine();

        System.out.print( "Entrez votre âge: " );
        int age = sc.nextInt();

        System.out.println( "Bonjour " + nom + ", vous avez " + age + " ans." );
        sc.close(); // Fermeture du Scanner
    }
}
```

Différences avec C

C	Java
printf()	System.out.printf()
scanf()	Scanner.nextXXX()
Gestion manuelle des buffers	Gestion automatique
Risques de débordement	Sécurité intégrée

Syntaxe de Base et Types de Données

Variables et Types :

- int, double, char, boolean, etc.

Les primitives doivent être déclarées par Visibilité :

type de primitive **identificateur** ;

Exemples :

```
int nombre = 10;
double prix = 19.99;
char lettre = 'A';
boolean estActif = true;
```

<i>Primitive</i>	<i>Étendue</i>	<i>Taille</i>
char	0 à 65 535	16 bits
byte	-128 à +127	8 bits
short	-32 768 à +32 767	16 bits
int	-2 147 483 648 à + 2 147 483 647	32 bits
long		64 bits
float	de $\pm 1.4E-45$ à $\pm 3.40282347E38$	32 bits
double		64 bits
boolean	true ou false	1 bit
void	-	0 bit

Pour respecter la typologie de java, les noms variables commencent toujours par une minuscule et pour indiquer un séparateur de mots, on utilise les majuscules.

int nbPersonnes;

String nomEtudiant;

- Les primitives de type numérique à pour valeur **0** par défaut.
- Les primitives de type boolean est initialisé par défaut à la valeur **false**.

Opérateurs en Java

✓ Opérateur d'affectation :

x = 3; // x reçoit 3.

w = 3;

x = y = z = w + 5 ; // z reçoit 8, y et x reçoit la valeur de z.

✓ Les opérateurs arithmétiques à deux opérandes :

+ : addition

- : soustraction

*** : multiplication**

/ : division % : modulo (reste de la division euclidienne).

TP 1 : Calcul du Prix Brut

Écrire programme qui calcule le *Prix Brut* (type *float*) d'un article à partir du *Prix Net* (type *float*) et du pourcentage de *TVA* (20 %), en utilisant la formule suivante :

$$PBRUT = PNET + PNET \cdot \frac{TVA}{100}$$

```
import java.util.Scanner;

public class TP1 {
    Run | Debug
    public static void main(String[] args) {
        float PN, PB;
        final int TVA = 20;
        System.out.println(x:"Donner le PrixNet : ");
        Scanner sc=new Scanner(System.in);
        PN = sc.nextFloat();
        PB = PN + (PN*TVA/100);
        System.out.println(x:"Le PrixBrut : ");
        System.out.println(PB);
    }
}
```

```
Donner le PrixNet :
100
Le PrixBrut :
120.0
PS C:\Users\User>
```


Opérateurs en Java

Opérateurs de **comparaison** : ==, !=, >, <, >=, <=

Opérateurs **logiques** : &&, ||, !

Les opérateurs arithmétiques à deux opérandes
(Les raccourcis)

`x = x + 4;` ou `x+=4;`
`z = z * y;` ou `z*=y;`
`v = v % w;` ou `v%=w;`

Les opérateurs relationnels:

- `==` : équivalent
- `<` : plus petit que
- `>` : plus grand que
- `<=` : plus petit ou égal
- `>=` : plus grand ou égal
- `!=` : non équivalent

Les opérateurs d'incrémentations et de décrémentation:

- `++` : Pour incrémenter (`i++` ou `++i`)
- `--` : Pour décrémentation (`i--` ou `--i`)

Les opérateurs logiques

- `&&` Et (deux opérandes)
- `||` Ou (deux opérandes)
- `!` Non (un seul opérande)

L'opérateur à trois opérandes ?:

- `condition ? expression_si_vrai : expression_si_faux`
- exemple : `x = (y < 5) ? 4 * y : 2 * y;`

Equivalent à :

```
if (y < 5)
    x = 4 * y;
else
    x = 2 * y;
```

Structures de Contrôle

Instructions conditionnelles : if, else, switch

■ L'instruction conditionnelle if

La syntaxe de l'instruction if peut être décrite de la façon suivante:

```
if (expression) instruction;
```

ou :

```
if (expression) {  
    instruction1;  
    instruction2;  
}
```

■ L'instruction conditionnelle else

```
if (expression) {  
    instruction1;  
}  
else {  
    instruction2;  
}
```

Exemple if :

```
if (nombre > 0) {  
    System.out.println('Nombre positif');  
} else {  
    System.out.println('Nombre négatif ou  
zéro');  
}
```

```
if (nombre > 0) {  
    System.out.println("Nombre positif");  
} else {  
    System.out.println("Nombre négatif ou zéro");  
}
```

Structures de Contrôle

■ Les instructions conditionnelles imbriquées

Java permet d'écrire ce type de structure sous la forme :

```
if (expression1) {  
    bloc1;  
}  
else if (expression2) {  
    bloc2;  
}  
else if (expression3) {  
    bloc3;  
}  
else {  
    bloc5;  
}
```

■ L'instruction switch

Syntaxe :

```
switch( variable) {  
    case valeur1: instr1;break;  
    case valeur2: instr2;break;  
    case valeurN: instrN;break;  
    default: instr;break;  
}
```

TP 2 : Maximum entre A et B

Écrire un programme JAVA qui lit deux valeurs entières (A et B) au clavier et qui affiche la plus grande des deux valeurs.

```
Entrez la première valeur (A) : 5
Entrez la deuxième valeur (B) : 10
La plus grande valeur est : 10
```

```
import java.util.Scanner;

public class TP2MaxAB {
    Run | Debug
    public static void main(String[] args) {
        // Créer un objet Scanner pour lire les entrées de l'utilisateur
        Scanner scanner = new Scanner(System.in);

        // Demander à l'utilisateur de saisir les deux valeurs
        System.out.print(s:"Entrez la première valeur (A) : ");
        int A = scanner.nextInt();

        System.out.print(s:"Entrez la deuxième valeur (B) : ");
        int B = scanner.nextInt();

        // Comparer les deux valeurs et afficher la plus grande
        if (A > B) {
            System.out.println("La plus grande valeur est : " + A);
        } else if (B > A) {
            System.out.println("La plus grande valeur est : " + B);
        } else {
            System.out.println(x:"Les deux valeurs sont égales.");
        }

        // Fermer le scanner
        scanner.close();
    }
}
```

TP 3 : Calculateur

```
char op = scanner.next().charAt(0);
```

Ecrire un programme JAVA qui permet de saisir un expression arithmétique composé :

- ✓ Deux entiers **A** et **B**,
- ✓ Une opération **OP** ('+', '-', '*', '/').

Puis, le programme affiche le résultat selon l'opération demandée.

```
Entrez le premier entier (A) : 9
Entrez le deuxième entier (B) : 5
Entrez l'opération ( + , - , * , / ) : *
Résultat : 45
```

```
Entrez le premier entier (A) : 10
Entrez le deuxième entier (B) : 5
Entrez l'opération ( + , - , * , / ) : /
Résultat : 2
```

```
Entrez le premier entier (A) : 5
Entrez le deuxième entier (B) : 0
Entrez l'opération ( + , - , * , / ) : /
Erreur : Division par zéro impossible.
```

```
Entrez le premier entier (A) : 13
Entrez le deuxième entier (B) : 4
Entrez l'opération ( + , - , * , / ) : ?
Opération invalide.
```

```
public class CalculatriceSimple {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print(s:"Entrez le premier entier (A) : ");
        int A = scanner.nextInt();
        System.out.print(s:"Entrez le deuxième entier (B) : ");
        int B = scanner.nextInt();
        System.out.print(s:"Entrez l'opération ( + , - , * , / ) : ");
        char op = scanner.next().charAt(0); // Lecture de l'opération

        switch (op) {
            case '+':
                System.out.println("Résultat : " + (A + B));
                break;
            case '-':
                System.out.println("Résultat : " + (A - B));
                break;
            case '*':
                System.out.println("Résultat : " + (A * B));
                break;
            case '/':
                if (B != 0) {
                    System.out.println("Résultat : " + (A / B));
                } else {
                    System.out.println(x:"Erreur : Division par zéro impossible.");
                }
                break;
            default:
                System.out.println(x:"Opération invalide.");
        }
    }
}
```

Structures de Contrôle

Boucles : for, while, do-while

■ La boucle for

La boucle **for** est une structure employée pour exécuter un bloc d'instructions un nombre de fois en principe connu à l'avance. Elle utilise la syntaxe suivante :

```
for (initialisation; test; incrémentation) {  
    instructions;  
}
```

Exemple :

```
for (int i = 2; i < 10; i++) {  
    System.out.println("I="+i);  
}
```

Structures de Contrôle

Boucles : for, while, do-while

■ L 'instruction While

```
while (condition){  
    BlocInstructions;  
}
```

■ L 'instruction do .. while

```
do{  
    BlocInstructions;  
}  
while (condition);
```

Exemple :

```
int s=0;int i=0;  
while (i<10){  
    s+=1;  
    i++;  
}  
System.out.println("Somme="+s);
```

Exemple :

```
int s=0;int i=0;  
do{  
    s+=1;  
    i++;  
}while (i<10);  
System.out.println("Somme="+s);
```

TP 4 : Calcul de la factorielle de N (N!)

Écrire un programme JAVA qui permet de calculer la factorielle $N! = 1 * 2 * 3 * ... * (N-1) * N$, d'un entier N en respectant que $0!=1$.

```
Entrez un entier N : 5
La factorielle de 5 est : 120
```

```
Entrez un entier N : 0
La factorielle de 0 est : 1
```

```
Entrez un entier N : -5
Erreur : la factorielle n'est définie pour N<0
```

```
public class TP4Factoriel {
    public static void main(String[] args) {
        // Créer un objet Scanner pour lire l'entrée de l'utilisateur
        Scanner scanner = new Scanner(System.in);

        // Demander à l'utilisateur de saisir un entier N
        System.out.print(s:"Entrez un entier N : ");
        int N = scanner.nextInt();

        // Calculer la factorielle de N
        // Utiliser long pour éviter les débordements pour des grandes valeurs de N
        long resultat = 1;

        // Si N est un entier positif ou nul, calculer la factorielle
        if (N >= 0) {
            for (int i = 1; i <= N; i++) {
                resultat *= i;
            }
            System.out.println("La factorielle de " + N + " est : " + resultat);
        } else {
            // Si N est négatif, afficher un message d'erreur
            System.out.println(x:"Erreur : la factorielle n'est définie pour N<0");
        }

        // Fermer le scanner
        scanner.close();
    }
}
```


TP 5 : Gestion des notes

Écrire un programme JAVA qui permet de saisir les notes pour une classe de 05 étudiants, puis affiche la moyenne des notes ?

```
Entrez la note de l'étudiant 1: 13
Entrez la note de l'étudiant 2: 12
Entrez la note de l'étudiant 3: 11
Entrez la note de l'étudiant 4: 16
Entrez la note de l'étudiant 5: 15
La moyenne des notes est : 13.4
```

```
import java.util.Scanner;
public class TP5Notes {
    Run | Debug
    public static void main(String[] args) {
        // Créer un objet Scanner pour lire l'entrée de l'utilisateur
        Scanner scanner = new Scanner(System.in);
        // Déclaration de la variable pour stocker la somme des notes
        double somme = 0;
        int nombreEtudiants = 05;
        // Saisie des notes pour les 20 étudiants
        for (int i = 1; i <= nombreEtudiants; i++) {
            System.out.print("Entrez la note de l'étudiant " + i + ": ");
            double note = scanner.nextDouble();
            somme += note; // Ajouter la note à la somme totale
        }
        // Calcul de la moyenne
        double moyenne = somme / nombreEtudiants;
        // Affichage de la moyenne
        System.out.println("La moyenne des notes est : " + moyenne);
        // Fermer le scanner
        scanner.close();
    }
}
```

Les tableaux

Les tableaux permettent de stocker plusieurs valeurs de même type dans une variable.

- Les valeurs contenues dans la variable sont repérées par un indice
- En langage java, les tableaux sont des objets

On peut définir un tableau sur tous les types java.

Syntaxe de définition d'un tableau :

type [] nom;

Déclaration :

- int tab [];
- String chaines[];

Exemple : Création d'un tableau

- tab = new int [20]; // tableau de 20 int
- chaines = new String [100]; // tableau de 100 chaine

Les tableaux

- Le nombre d'éléments du tableau est mémorisé. Java peut ainsi détecter à l'exécution le dépassement d'indice et générer une exception. Mot clé « **length** »
 - Il est récupérable par : **nomTableau.length**
`int taille = tab.length; //taille vaut 20`

Exemple :

```
int [ ] tab = new int[50];  
System.out.println("Taille du tableau " + tab.length );
```

Comme en C/C++, les indices d'un tableau commencent à '0'. Donc un tableau de taille 100 aura ses indices qui iront de 0 à 99.

Initialisation :

```
tab[0]=1;  
tab[1]=2; //etc.  
noms[0] = new String( "Boule");  
noms[1] = new String( "Bill");//etc
```

Création et initialisation simultanées :

```
String noms [ ] = {"Boule","Bill"};  
Point pts[ ] = { new Point (0, 0), new Point (10, -1)};
```

Les tableaux

Tab1.java

```
public class Tab1
{
    public static void main (String args[])
    {
        int tab[ ] ;
        tab = new int[4];
        tab[0]=5;
        tab[1]=3;
        tab[2]=7;
        tab[3]=tab[0]+tab[1];
    }
}
```

Pour déclarer une variable tableau on indique le *type* des éléments du tableau et le *nom de la variable tableau* suivi de `[ ]`

on utilise `new <type> [taille];` pour initialiser le tableau

On peut ensuite affecter des valeurs au différentes cases du tableau :
`<nom_tableau>[indice]`

Les indices vont toujours de 0 à (taille-1)

Les tableaux

Tab1.java

```
public class Tab1
{
    public static void main (String args[])
    {
        int tab[ ] ;
        tab = new int[4];
        tab[0]=5;
        tab[1]=3;
        tab[2]=7;
        tab[3]=tab[0]+tab[1];
    }
}
```

Mémoire

0x258

0
0
0
0

Les tableaux

Tab1.java

```
public class Tab1
{
    public static void main (String args[])
    {
        int tab[ ];
        tab = new int[4];
        tab[0]=5;
        tab[1]=3;
        tab[2]=7;
        tab[3]=tab[0]+tab[1];
    }
}
```

Mémoire

0x258

5

3

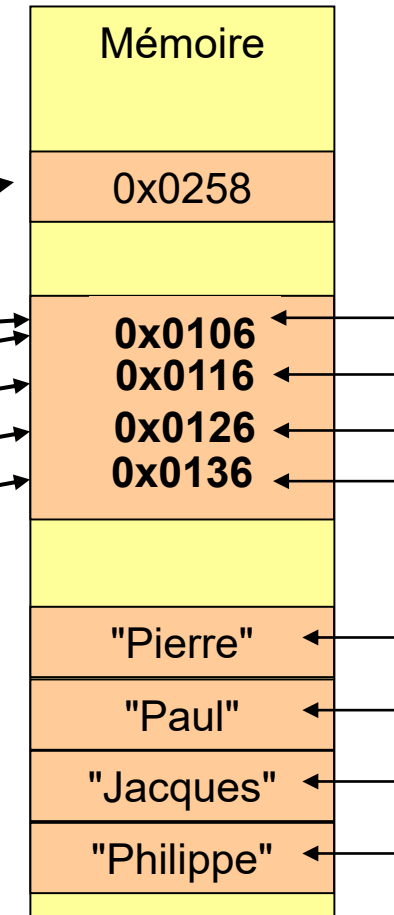
7

8

Les tableaux

Tab2.java

```
public class Tab2
{
    public static void main (String args[])
    {
        String tab[ ] ;
        tab = new String[4];
        tab[0]=new String("Pierre");
        tab[1]=new String("Paul");
        tab[2]=new String("Jacques");
        tab[3]=new String("Philippe");
    }
}
```



Méthodes

Une fonction d'une classe est appelée « méthode ».

En java une méthode peut retourner une valeur et avoir une liste de paramètres :

```
type_de_retour nom_de_méthode ( liste_des_paramètres )  
{  
    corps de la méthode  
}
```

Exemple : Une méthode qui permet de retourner la somme de deux entiers :

```
int addition( int a, int b )  
{  
    return a + b;  
}
```

Le mot clef « return » permet de renvoyer une valeur

TP 6 : Gestion des notes

Reprends l'exercice de la gestion des notes (TP5), en écrivant un programme JAVA qui permet de gérer les notes de 30 étudiants au Max :

- écrire une méthode *saisie* qui permet de saisir la note d'un seul étudiant ?
- écrire une méthode *moyenne* qui permet de calculer la moyenne des notes ?
- écrire une méthode *trier* qui permet de trier le tableau des notes dans un sens décroissant ?

Chapitre 2 : POO en Java

-  Concepts de la POO
-  Classes et objets
-  Encapsulation
-  Héritage
-  Polymorphisme
-  Interfaces
-  Gestion des exceptions
-  Collections Java
-  Manipulation de fichiers

Élément 1 : Fondamentaux et POO en Java

Concepts de la POO

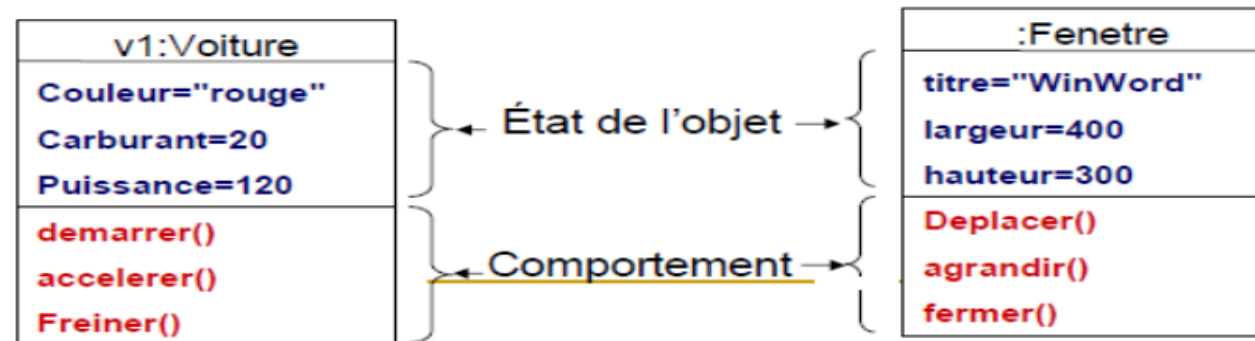
- **Définition** : Paradigme de programmation basé sur les objets qui encapsulent des données et des comportements.
- **Principes fondamentaux** :
 - **Encapsulation** : Regrouper les données et les méthodes dans une classe.
 - **Héritage** : Réutiliser et étendre les fonctionnalités existantes.
 - **Polymorphisme** : Adapter le comportement en fonction du contexte.
 - **Abstraction** : Cacher les détails complexes pour ne montrer que l'essentiel.

Classes et Objets

Objet

- **Un objet est une structure informatique définie par un état et un comportement.**
- **Objet = état + comportement.**
- **L'état regroupe les valeurs instantanées de tous les attributs de l'objet.**
- **Le comportement regroupe toutes les actions et les réactions de l'objet.**
- **L'état d'un objet peut changer dans le temps.**
- **Généralement , c'est le comportement qui modifie l'état de l'objet.**

Exemple:



Classes et Objets

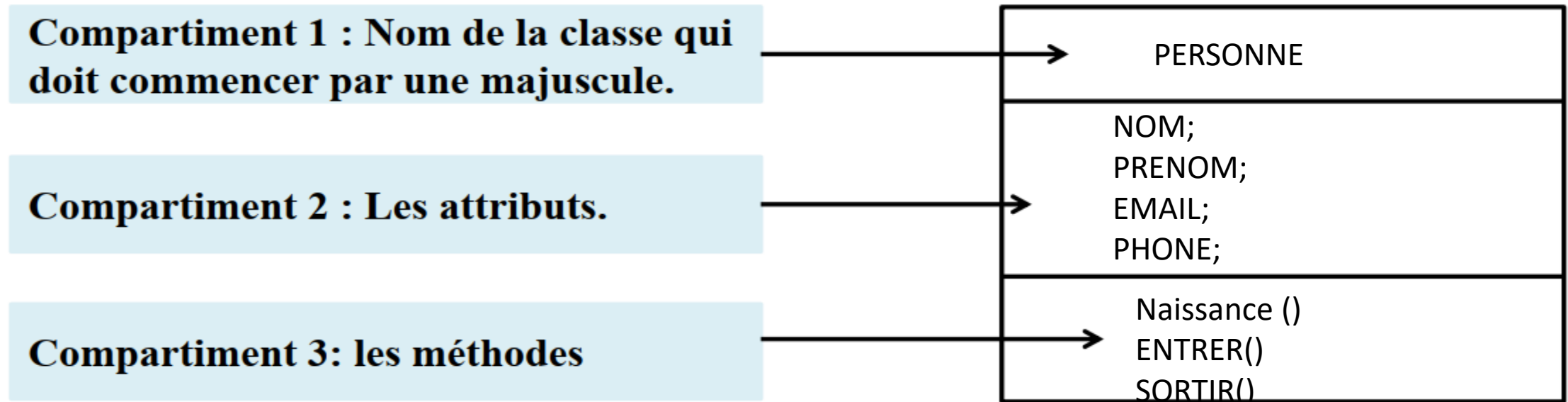
Classe

- **Les objets qui ont des caractéristiques communes sont regroupés dans une entité appelé classe.**
- **Alors la classe décrit le domaine de définition d'un ensemble d'objets.**
- **Chaque objet appartient à une classe.**
- **La classe est la forme générale des objets.**
- **Les objets sont construit a partir de leur par un processus qui s'appelle l'instanciation.**
- **Alors tout objet est une instance d'une classe.**

Classes et Objets

Classe

- Une classe est généralement représenté par un rectangle qui comporte trois compartiments



Classes et Objets

Classe

Attribut:

- **Les attributs et les opérations sont les propriétés d'une classe. Les attributs décrivent l'état de l'objet de la classe. Un attribut est déclaré selon le format suivant:**

Visibilité TypeAttribut nomAttribut [Multiplicité] = [ValeurInit]

- ☐ **Visibilité:** + (Public), - (Privé), # (Protégé), par défaut la Visibilité est protected.
- ☐ **nomAttribut:** Le nom de l'attribut doit être unique et commencer par un minuscule.
- ☐ **Multiplicité:** nombre d'attributs de ce type (Exemple tableau []).
- ☐ **TypeAttribut:** le type de l'attribut (type primitif ou classe).
- ☐ **ValeurInit:** la valeur initiale de l'attribut.

Classes et Objets

Classe

Méthode:

- **Les méthodes décrivent le comportement des objets. Elles représentent les fonctions qu'une classe sait exécuter.**
- **Les méthodes appelées également méthodes dans les langages objets.**
- **Syntaxe UML pour la définition d'une opération:**

Visibilité TypeDeRetour nomOperation (Paramètres)

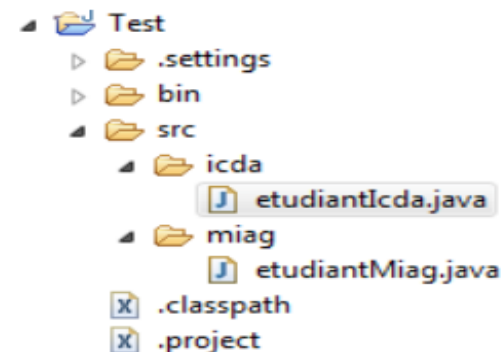
- ☐ **Visibilité:** + (public), - (privée) ou # (protégée).
- ☐ **TypeDeRetour:** le type de la valeur de retour de l'opération (type primitif ou classe)
- ☐ **nomOperation:** nom de la méthode
- ☐ **Paramètres:** les paramètres de la méthode.

Classes et Objets

Classe

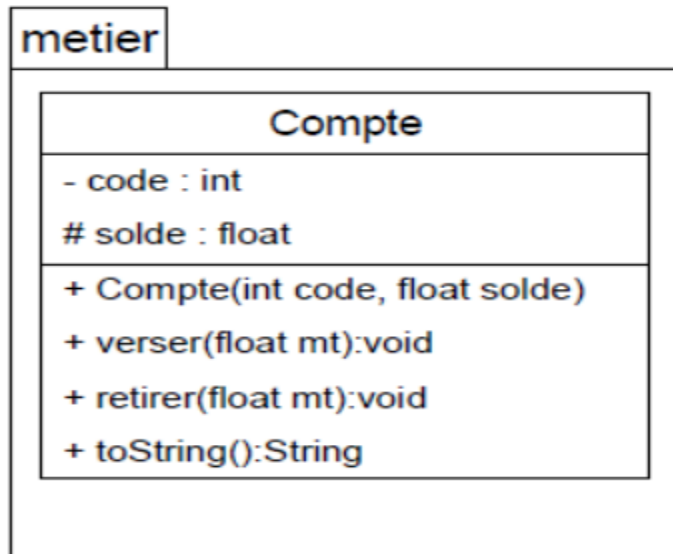
- **Les classes sont généralement regroupées dans des packages.**
- **Chaque package est représenté graphiquement par un dossier qui peut comporter un ou plusieurs classes.**
- **Les packages offrent un mécanisme efficace pour organiser le développement d'un projet informatique.**
- **Les packages divisent et organisent les modules d'un projet de la même manière que les dossiers organisent le système de fichiers.**

Structure d'un projet java:



Classes et Objets

Exemple d'implémentation d'une classe avec java



```

package metier;
public class Compte {
    // Attributs
    private int code;
    protected float solde;
    // Constructeur
    public Compte(int c,float s){
        code=c;
        solde=s;
    }
    // Méthode pour verser un montant
    public void verser(float mt){
        solde+=mt;
    }
    // Méthode pour retirer un montant
    public void retirer(float mt){
        solde-=mt;
    }
    // Une méthode qui retourne l'état du compte
    public String toString(){
        return (" Code="+code+" Solde="+solde);
    }
}

```

Classes et Objets

Création des Objets dans java:

- Dans java, pour créer un objet d'une classe, on utilise la commande **new** suivie par un constructeur de la classe.
- Les constructeurs de la classe sont des méthodes spéciales d'une Visibilité public, qui porte le même nom que la classe et ils ont pas un type de retour.
- Il existe plusieurs types des constructeurs:
 - ❑ Constructeur par défaut: toute classe à un constructeur par défaut qui ne reçoit pas des paramètres .
 - ❑ Constructeur avec des paramètres: définit par le programmeur s'il est nécessaire. Exemple

```
public etudiantIcda(String nom, String prenom) {  
    super();  
    this.nom = nom;  
    Prenom = prenom;  
}
```

Classes et Objets

Création des Objets dans java:

- ❑ Constructeur synthétisé: une fois le programmeur déclare un constructeur avec paramètres le constructeur par défaut n'est plus exploitable, alors le programme déclare un constructeur vide nommé synthétisé.
- Exemple de la syntaxe:

```
public static void main(String[] args) {
    Compte c1=new Compte(1,5000);
    Compte c2=new Compte(2,6000);
    c1.verser(3000);
    c1.retirer(2000);
    System.out.println(c1.toString());
}
```

C1 : code = 1, Solde =5000

C2 : code = 2, Solde =6000

C1 : code = 1, Solde = 8000 : 5000+3000

Code=1 Solde= 6000

c1:Compte
code=1
solde=6000
verser(float mt)
retirer(float mt)
toString()

c2:Compte
code=2
solde=6000
verser(float mt)
retirer(float mt)
toString()

Encapsulation

Encapsulation (Accessibilité):

Dans java, il existe 4 niveaux de la protection:

- ❑ **Private (-)** : Un membre privé d'une classe n'est accessible qu'à l'intérieur de cette classe.
- ❑ **Protected (#)** : un membre protégé d'une classe est accessible à :
 - L'intérieur de la classe.
 - Aux classes dérivées de cette classe.
 - Aux classes de même package.
- ❑ **Public (+)**: accès à partir de toute entité interne et externe à la classe.

RQ : Par défaut l'autorisation est de package. Cette autorisation indique que uniquement les classes du même package ont l'autorisation d'accès.

Encapsulation

Encapsulation (Accessibilité): Getters et Setters

- Les attributs privés d'une classe ne sont pas accessibles qu'à l'intérieur de la classe.
- Pour donner la possibilité à autres classes d'accéder à ces attributs il faut définir dans la classe des méthodes publiques qui permettant de:

- ☐ Lire la variable: **accesseurs** ou **Getters**.

- ☐ Modifier la variable: **mutateurs** ou **Setters**.

- Exemple de la déclaration des Getters:

```
public String getNom() {  
    return nom;  
}
```

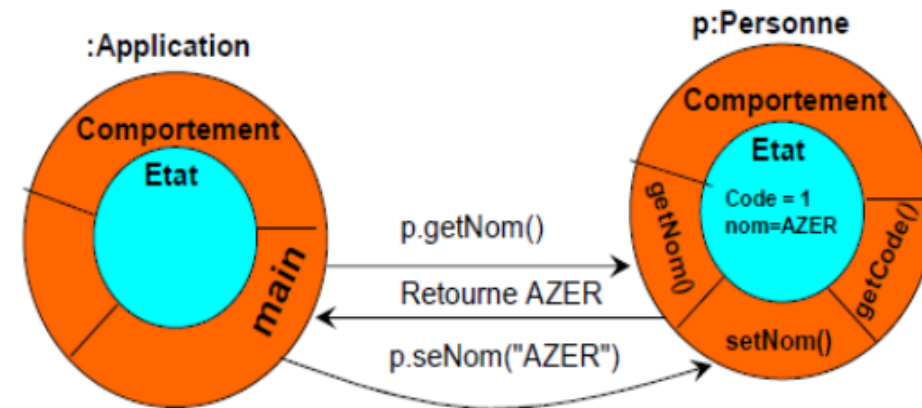
- Exemple de la déclaration des Setters:

```
public void setNom(String nom) {  
    this.nom = nom;  
}
```


Encapsulation

Encapsulation (Accessibilité):

```
public class Application {  
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.setNom("AZER");  
        System.out.println(p.getNom());  
    }  
}
```



- Généralement l'état d'un objet est privé ou protégé et son comportement est public.
- Quand l'état de l'objet est privé Seules ces méthodes qu'ont le droit d'y accéder.
- Quand l'état de l'objet est protégé, les méthodes des classes dérivées et les classes appartenant au même package peuvent également y accéder .

TP 7 : Classe étudiant

Gestion des Étudiants

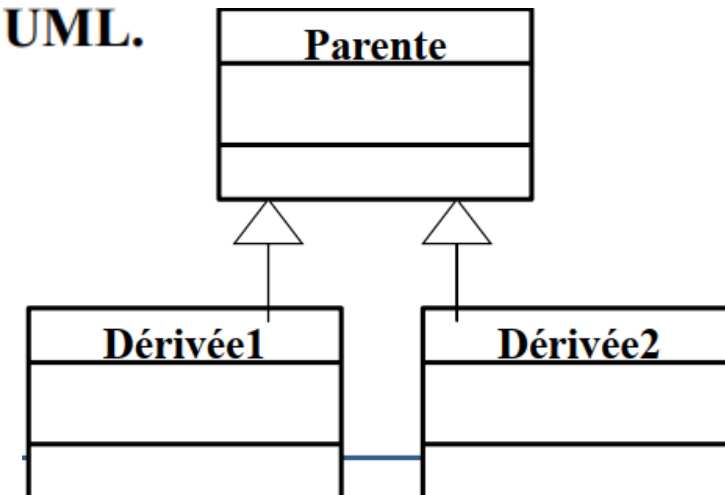
1. Créez une **classe** appelée *Etudiant* qui possède les attributs suivants :
 - **nom** (chaîne de caractères)
 - **age** (entier)
 - **moyenne** (double)
2. La classe doit inclure les fonctionnalités suivantes :
 - Un **constructeur** pour initialiser les attributs.
 - Une méthode **afficherDetails** qui affiche les informations de l'étudiant.
 - Une méthode **estAdmis** qui retourne **true** si la moyenne est supérieure ou égale à **10**, sinon **false**.
3. Dans la méthode main, faites les étapes suivantes :
 - Créez plusieurs objets de la classe *Etudiant*.
 - Affichez leurs détails.
 - Déterminez si chaque étudiant est admis ou non.

Héritage

- Dans la programmation orientée objet, l'héritage offre un moyen très efficace qui permet la réutilisation du code.
- En effet une classe peut hériter d'une autre classe des attributs et des méthodes et non pas **les constructeurs**.
- L'héritage, quand il peut être exploité, fait gagner beaucoup de temps en terme de développement et en terme de maintenance des applications.
- La réutilisation du code fut un argument déterminant pour vendre les méthodes orientées objets.
- L'héritage en java est traduit par le qualificateur **extends**.
- Toutes les classes héritent implicitement la classe **Object**.
- Une classe ne peut hériter explicitement qu'une seule classe.

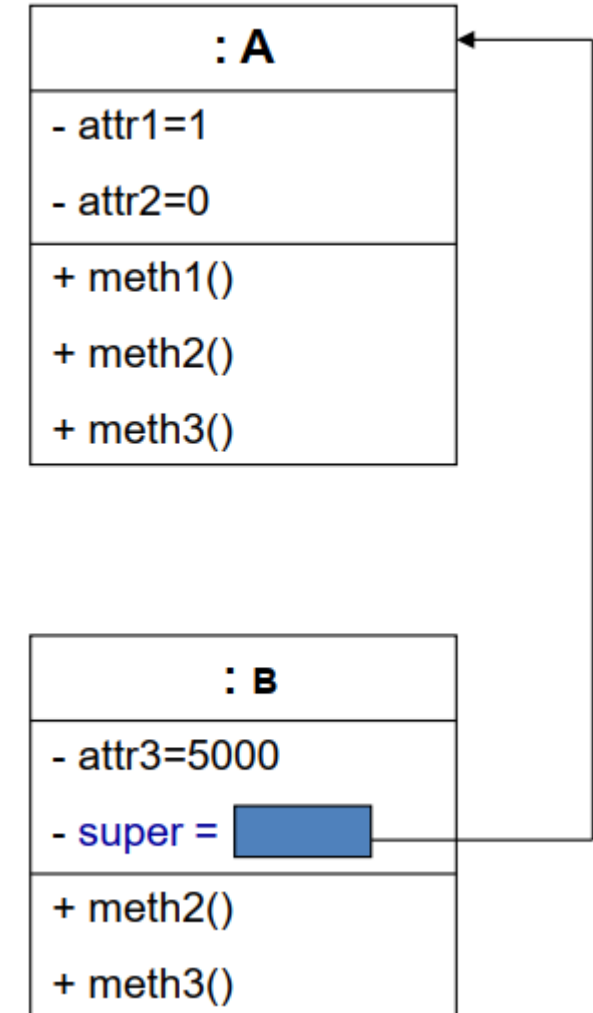
Héritage

- Si la classe A hérite directement de la classe B alors elle hérite indirectement la classe Object.
- Le constructeur de la classe **dérivée** peut faire appel au constructeur de la classe **parente** en utilisant le mot **super()** suivi de ses paramètres.
- La classe dérivée peut redéfinir les méthodes de la classe parente comme elle peut appeler ces méthodes via le qualificateur **super()**.
- L'héritage est traduit par une association de type généralisation en UML.



Héritage

- **Quand on crée une instance d'une classe, la classe parente est automatiquement instanciée et l'objet de la classe parente est associé à l'objet créé à travers la référence « super » injectée par le compilateur**
- **Lors de l'instanciation, l'héritage entre les classes est traduit par une composition entre un objet de la classe instanciée et d'un objet de la classe parente qui est créé implicitement.**



TP 8 : Gestion des employés d'une entreprise

Créez une classe **Employe** avec les attributs suivants :

- ✓ nom (type String)
- ✓ prenom (type String)
- ✓ salaire (type double)

Cette classe doit comporter :

- ✓ Un constructeur pour initialiser ses attributs.
- ✓ Une méthode `afficherDetails()` qui affiche les informations de l'employé.

2. Créez une classe **Manager** qui hérite de la classe **Employe** et ajoute un nouvel attribut :

- ✓ prime (type double) représentant une prime de performance.

Cette classe doit comporter :

- ✓ Un constructeur qui initialise tous les attributs (y compris ceux hérités).
- ✓ Une méthode `afficherDetails()` qui surcharge la méthode de la classe mère pour afficher aussi la prime.

3. Dans une classe principale **Main**, créez un tableau d'employés contenant à la fois des objets de type **Employe** et **Manager**, et affichez leurs détails.

Tools for Java > 25-26

Nom

JavaCodingPack-0.4.2.exe

jdk-25_windows-x64_bin.exe

VSCodeUserSetup-x64-1.105.1.exe

C:\WINDOWS\system32\cmd.exe

```
Microsoft Windows [version 10.0.16299.1127]
(c) 2017 Microsoft Corporation. Tous droits réservés.
```

```
C:\Users\User>java --version
java 23.0.1 2024-10-15
Java(TM) SE Runtime Environment (build 23.0.1+11-39)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.1+11-39, mixed mode, sharing)
```

```
C:\Users\User>java --version
java 25.0.1 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 25.0.1+8-LTS-27)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.1+8-LTS-27, mixed mode, sharing)
```

Visual Studio Code

Docs Updates Blog API Extensions MCP FAQ

Overview

SETUP

GET STARTED

CONFIGURE

EDIT CODE

BUILD, DEBUG, TEST

GITHUB COPILOT

SOURCE CONTROL

TERMINAL

LANGUAGES

Overview

JavaScript

JSON

HTML

Install Visual Studio Code for Java

To help you set up quickly, we recommend you use the **Coding Pack for Java**, which is the bundle of VS Code, the Java Development Kit (JDK), and a collection of suggested extensions by Microsoft. The Coding Pack can also be used to fix an existing development environment.

[Install the Coding Pack for Java - Windows](#)

[Install the Coding Pack for Java - macOS](#)

Note: The Coding Pack for Java is only available for Windows and macOS. For other operating systems, you will need to manually install a JDK, VS Code, and Java extensions.

If you have already installed VS Code and want to add Java support to it, we recommend using the

[Coding Pack for Java](#)

Install Coding Pack for Java

This Setup Wizard will automatically download and install Visual Studio Code and a JDK distribution on your behalf.

The JDK binary to be installed is provided by AdoptOpenJDK, free and [open source licensed](#). By downloading and using Visual Studio Code, you agree to [Microsoft's license terms](#). The Microsoft and third party extensions in the Java Extension Pack are licensed under these [licenses](#). The third party software is licensed to you by the third parties under the accompanying licenses.

By continuing, you agree to these licenses.

☒ I accept the terms in the License Agreement

Cancel

Next

Install Coding Pack for Java

This Setup Wizard will automatically download and install Visual Studio Code and a JDK distribution on your behalf.

The JDK binary to be installed is provided by AdoptOpenJDK, free and [open source licensed](#). By downloading and using Visual Studio Code, you agree to [Microsoft's license terms](#). The Microsoft and third party extensions in the Java Extension Pack are licensed under these [licenses](#). The third party software is licensed to you by the third parties under the accompanying licenses.

By continuing, you agree to these licenses.

☒ I accept the terms in the License Agreement

Cancel

Next

Configure Components

Required components are listed below. Click Install to proceed.

Java Development Kit:	25	OK
Visual Studio Code:	1.105.1	OK

Actions:

- Install Java Extensions

Cancel

Back

Install

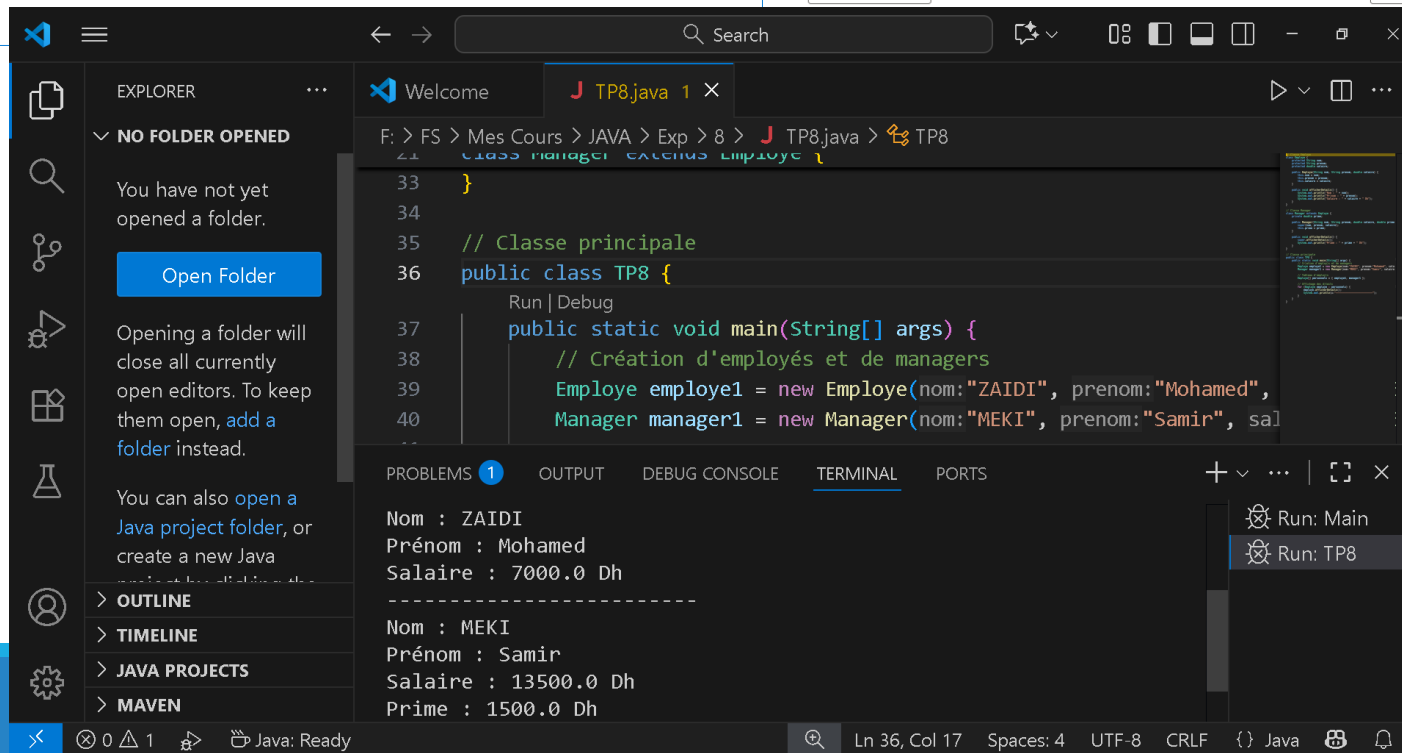
Completing the Setup Wizard

The Wizard has finished installing all the components on your computer. The application may be launched by selecting the installed icons.

Click Finish to exit the Wizard.

☒ Launch Visual Studio Code

Finish



Attribut et méthode statique d'une classe

- Les objets d'une même classes peuvent partager des mêmes variables qui sont stockées au niveau de la classe. Ce genre de variables, s'appellent les **variables statiques** ou **variables de classes**.
- Alors un attribut statique d'une classe est un attribut qui appartient à la classe et partagé par tous les objets de cette classe.
- Comme un attribut une méthode peut être déclaré statique.

Compte
- code : int
solde : float
- <u>nbComptes</u> :int
+ Compte(float solde)
+ verser(float mt):void
+ retirer(float mt):void
+ toString():String
+ <u>getNbComptes</u> ()int

Attribut et méthode statique d'une classe

La variable nbComptes permet de stocker le nombre de comptes créés

La variable nbComptes est commun entre les objets alors il est déclaré static

Compte
- code : int
solde : float
- <u>nbComptes</u> :int
+ Compte(float solde)
+ verser(float mt):void
+ retirer(float mt):void
+ toString():String
+ <u>getNbComptes()</u> :int

```

package metier;
public class Compte {
    // Variables d'instances
    private int code;
    private float solde;
    // Variable de classe ou statique
    private static int nbComptes;
    public Compte(float solde){
        this.code=++nbComptes;
        this.solde=solde;
    }
    // Méthode pour verser un montant
    public void verser(float mt){
        solde+=mt;
    }
    // Méthode pour retirer un montant
    public void retirer(float mt){
        solde-=mt;
    }
    // retourne l'état du compte
    public String toString(){
        return(" Code="+code+" Solde="+solde)
    }
    // retourne la valeur de nbComptes
    public static int getNbComptes(){
        return(nbComptes);
    }
}
  
```


Surcharge en Java

- Dans une classe, on peut définir plusieurs constructeurs. Chacun ayant une signature différentes (paramètres différents).
- On dit que le constructeur est surchargé
- On peut également surcharger une méthode. Cela peut dire qu'on peut définir, dans la même classe plusieurs méthodes qui ont le même nom et des signatures différentes;
- La signature d'une méthode désigne la liste des arguments avec leurs types.
- Un constructeur peut appeler un autre constructeur de la même classe en utilisant le mot **this()** avec des paramètres éventuels

Classe abstraite

- Une classe abstraite est une classe qui ne peut pas être instanciée.
- Une classe abstraite est généralement créée pour en faire dériver de nouvelle classe par héritage.

- Exemple:

```
public abstract class Compte {  
    private int code;  
    protected float solde;  
    private static int nbComptes;  
  
    // Constructeurs  
    // Méthodes  
}
```

Classe abstraite : *Méthode abstraite*

- Une méthode abstraite peut être déclarée à l'intérieur d'une classe abstraite.
- Une méthode abstraite est une méthode qui n'a pas de définition.
- Une méthode abstraite est une méthode qui doit être redéfinie dans les classes dérivées.

Exemple:

```
public abstract class Compte {  
    // Membres  
    ...  
    // Méthode abstraite  
    public abstract void afficher();  
}
```

```
public class CompteSimple extends Compte {  
    // Membres  
    ...  
    public void afficher() {  
        System.out.println("Solde="+solde+" Découvert="+decouvert);  
    }  
}
```

Interfaces

- Une interface est une sorte de classe abstraite qui ne contient que des méthodes abstraites.
- Dans java une classe **hérite** d'une seule classe et peut **implémenter** en même temps de plusieurs interface.
- Une interface IA peut implémenter une autre interface IB.
- Alors toute classe qui implémente l'interface IA implémente aussi l'interface IB.
- Une classe qui implémente une interface doit redéfinir toutes les méthodes de cette interface.

Interfaces

- **Exemple de déclaration d'une interface:**

```
public interface Solvable {  
    public void solver();  
    public double getSolde();  
}
```

- **Exemple d'implementation d'une interface:**

```
public class CompteSimple extends Compte implements Solvable {  
    private float decouvert;  
    public void afficher() {  
        System.out.println("Solde="+solde+" Découvert="+decouvert);  
    }  
    public double getSolde() {return solde;}  
    public void solver() {this.solde=0;}}
```

Classe de type final

- Une classe de type final est une classes qui ne peut pas être dérivée.
- Exemple de syntaxe:

La classe CompteSimple peut être déclarée final en écrivant:

```
public final class CompteSimple extends Compte {  
    private float decouvert;  
    public void afficher() {  
        System.out.println("Solde="+solde+"  
        Découvert="+decouvert);  
    }  
}
```

Variable & méthode : *final*

- Une variable **final** est une variable dont la valeur ne peut pas changer. Autrement dit, c'est une constante:
- Exemple : **final double** PI=3.14;
- Une méthode **final** est une méthode qui ne peut pas être redéfinie dans les classes dérivées.
- Exemple : La méthode verser de la classe suivante ne peut pas être redéfinie dans les classes dérivées car elle est déclarée **final**

```
public class Compte {  
    private int code; protected float solde;  
    private static int nbComptes;  
  
    public final void verser(float mt) {  
        solde+=mt;  
    }  
    public void retirer(float mt) {  
        if(mt<solde) solde-=mt;  
    }  
}
```


Polymorphisme

- ✓ Le **polymorphisme** signifie littéralement

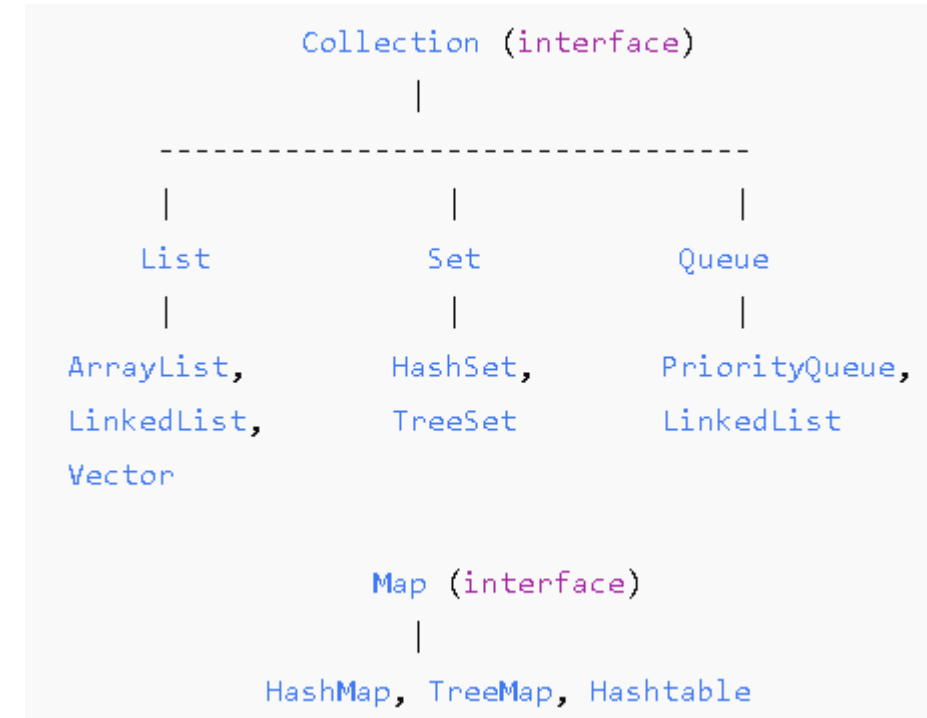
plusieurs formes

- ✓ En Java, il permet à **une même méthode** ou **une même interface** d'avoir **plusieurs comportements différents** selon le contexte ou le type d'objet qui l'utilise.
- ✓ Une même action (méthode) peut se comporter différemment selon l'objet qui l'invoque.

Type	Description	Moment d'application
Polymorphisme de compilation (ou statique)	Se fait grâce à la surcharge de méthodes (méthodes avec le même nom mais paramètres différents).	Pendant la compilation
Polymorphisme d'exécution (ou dynamique)	Se fait grâce à la redéfinition (override) de méthodes dans les classes dérivées.	Pendant l'exécution

Collection

- ✓ Une **Collection** en Java est un **ensemble d'objets regroupés dans une même structure de données**.
- ✓ La structure de collections fournit un ensemble d'interfaces et de classes permettant : le stockage dynamique d'objets, le tri, la recherche, la suppression, et la parcour à l'aide d'itérateurs.
- ✓ Les collections remplacent avantageusement *les tableaux (arrays)* qui ont une taille fixe



Collection : List

- ✓ Une List stocke les éléments dans un ordre précis, et peut contenir des doublons.

Implémentations :

- ✓ **ArrayList** → rapide en lecture, lente en insertion au milieu.
- ✓ **LinkedList** → rapide en insertion/suppression, lente en accès aléatoire.
- ✓ **Vector** → similaire à ArrayList mais synchronisé (ancien).

```
import java.util.*;
public class ExempleList {
    public static void main(String[] args) {
        List<String> noms = new ArrayList<>();
        noms.add("Alice");
        noms.add("Bob");
        noms.add("Charlie");
        noms.add("Alice"); // doublon autorisé
        System.out.println("Liste : " + noms);
        System.out.println("Élément à l'indice 1 : " + noms.get(1));
    }
}
```

Collection : Set

- ✓ Un **Set** ne contient aucun doublon et n'a pas d'ordre fixe (sauf **TreeSet**).

Implémentations :

- ✓ **HashSet** → rapide, ordre non garanti.
- ✓ **LinkedHashSet** → conserve l'ordre d'insertion.
- ✓ **TreeSet** → trie automatiquement les éléments.

```
import java.util.*;
public class ExempleSet {
    public static void main(String[] args) {
        Set<String> fruits = new HashSet<>();
        fruits.add("Pomme");
        fruits.add("Orange");
        fruits.add("Banane");
        fruits.add("Orange"); // ignoré
        System.out.println("Set : " + fruits);
    }
}
```

Collection : Map

- ✓ Une **Map** stocke des paires (*clé*, *valeur*).
- ✓ Chaque **clé** est unique, mais plusieurs clés peuvent avoir la même **valeur**.

Implémentations :

- ✓ **HashMap** → rapide, ordre non garanti.
- ✓ **LinkedHashMap** → conserve l'ordre d'insertion.
- ✓ **TreeMap** → trie les clés automatiquement

```
import java.util.*;
public class ExempleMap {
    public static void main(String[] args) {
        Map<Integer, String> etudiants = new HashMap<>();
        etudiants.put(1, "Alice");
        etudiants.put(2, "Bob");
        etudiants.put(3, "Charlie");
        System.out.println("Map des étudiants : " + etudiants);
        System.out.println("Nom de l'étudiant 2 : " + etudiants.get(2));
    }
}
```

Collection : Parcourir et Manipuler

Avec for-each

```
java

for (String nom : noms) {
    System.out.println(nom);
}
```

Avec un Iterator

```
java

Iterator<String> it = noms.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Méthode	Description
add(E e)	Ajoute un élément
remove(Object o)	Supprime un élément
clear()	Vide la collection
size()	Renvoie le nombre d'éléments
contains(Object o)	Vérifie la présence d'un élément
isEmpty()	Vérifie si la collection est vide

Collection : Parcourir et Manipuler

Avantages des Collections :

- ✓ Manipulation flexible de données
- ✓ Structures de données réutilisables
- ✓ Tri et recherche facilités (Collections.sort(), contains())
- ✓ Compatible avec les flux (Streams) introduits en Java 8

Type	Caractéristique principale	Doublons	Ordre
List	Liste ordonnée d'éléments	Oui	Oui
Set	Ensemble unique d'éléments	Non	Non (sauf TreeSet)
Map	Paires clé/valeur	(clés uniques)	Variable selon implémentation

Gestion des exceptions

- ✓ Une exception est une interruption de l'exécution d'un programme suite à une erreur.
- ✓ Les exceptions représentent le mécanisme de gestion des erreurs intégré au langage Java.
- ✓ Pour gérer l'exception, on peut utiliser le bloc **try** et **catch** entourant le code qui peut provoquer l'exception.
- ✓ Par exemple : Une division par zéro provoque une exception de type *ArithmeticException*.

```
public class Exemple {
    public static void main(String[] args) {
        int res = 10 / 0;
        System.out.println("Valeur de res est : " + res);
    }
}
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Exemple.main(Exemple.java:4)
```

```
try
{
    //statement1;
    //statement2;
    //statement3;
    //...
}
catch(exception-type)
{
    //code that handles exception.
}
```

Enclosing statements in try block may throw an exception.

catch block catches and handles a type of an exception.

```
public class Exemple {
    public static void main(String[] args) {
        try {
            int res = 10/0;
        } catch (ArithmeticException e) {
            e.printStackTrace();
            System.out.println("NumberFormatException est traité");
        }
        System.out.println("Continuez votre code ici");
    }
}
```

```
NumberFormatException est traité
Continuez votre code ici
```

Manipulation de fichiers

- ✓ La création et l'écriture de fichiers en Java est une tâche fondamentale pour de nombreuses applications.
- ✓ Java fournit plusieurs classes et méthodes pour gérer efficacement les opérations sur les fichiers.
- ✓ La **classe File Java** permet de gérer les fichiers, les noms de chemins de fichiers et de répertoires de manière abstraite. Cette classe est utilisée pour la création de fichiers et de répertoires, la recherche de fichiers, la suppression de fichiers, etc.



Manipulation de fichiers : Création

- ✓ Pour créer un fichier en Java, c'est à l'aide de la classe **File** et la méthode **createNewFile()**.
- ✓ Cette méthode crée un fichier vide, s'il n'existe pas à l'emplacement spécifié et renvoie **true**. Si le fichier est déjà présent, cette méthode renvoie **false**.
- ✓ **Exemple :** Créer un simple fichier texte
 1. Importer les bibliothèques **File** et **IOException** qui se trouvent dans le **package java.io**
 2. Instanciation sur la **classe File** pour créer un nouveau **objet myFile** du type **file**
 3. Crée le fichier avec la méthode : **createNewFile()** (**myFile.createNewFile()**)
- ✓ **IOException** : Si une erreur d'entrée / sortie survient lors de la création du fichier.

```
import java.io.File;
import java.io.IOException;
public class NewFile
{
    public static void main( String[] args )
    {
        try {
            File myFile = new File("F:/FS/Mes Cours/JAVA/Exp/9 File/newFile.txt");
            /* Si le fichier est créé, alors la méthode createNewFile() renverrait
               true et si le fichier est déjà présent il retournerait faux */
            boolean varF = myFile.createNewFile();
            if (varF){
                System.out.println("Fichier créé avec succès");
            }
            else{
                System.out.println("Le fichier existe déjà");
            }
        } catch (IOException e) {
            System.out.println("Exception Occurred:");
            e.printStackTrace();
        }
    }
}
```

Manipulation de fichiers : Écriture

- ✓ La classe *FileWriter* permet d'ouvrir un flux d'écriture sur un fichier via son constructeur :

`FileWriter( String nom_du_fichier, boolean append )`

- ✓ Si la valeur du `boolean append` est mis à `false`, le contenu du fichier sera écrasé par le nouveau contenu
- ✓ Si la valeur du `boolean append` est mis à `true`, le nouveau contenu sera ajouté à l'ancien

```
import java.io.FileWriter;
import java.io.IOException;
public class WriteFile
{
    public static void main( String[] args )
    {
        try
        {
            String filename= "F:/FS/Mes Cours/JAVA/Exp/9 File/newFile.txt";
            FileWriter fw = new FileWriter(filename,true);
            //la valeur true entraine l'ajout du nouveau contenu à l'ancien
            fw.write("Voici le contenu qui va être ajouté au fichier ");
            //appends the string to the file
            fw.close();
        }
        catch(IOException ioe)
        {
            System.err.println("IOException: " + ioe.getMessage());
        }
    }
}
```

Manipulation de fichiers : Lecture

- ✓ **FileReader**: Utilisé pour lire des fichiers texte de manière simple et efficace.
- ✓ **BufferedReader**: S'enroule autour de *FileReader* pour lire du texte à partir d'un flux d'entrée de caractères, en mettant les caractères en mémoire tampon pour une lecture efficace des caractères, des tableaux et des lignes.

```
PS F:\FS\Mes Cours\JAVA\Exp\9 File> javac ReadFile.java
PS F:\FS\Mes Cours\JAVA\Exp\9 File> java ReadFile
Voici le contenu qui va être ajouté au fichier
```

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class ReadFile
{
    public static void main( String[] args )
    {
        String filename= "F:/FS/Mes Cours/JAVA/Exp/9 File/newFile.txt";
        try (FileReader fileReader = new FileReader(filename);
            BufferedReader bufferedReader = new BufferedReader(fileReader)) {
            String line;
            while ((line = bufferedReader.readLine()) != null) {
                System.out.println(line);
            }
        }
        catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

Manipulation de fichiers : Supprimer

- ✓ Pour supprimer un fichier existant en Java, il suffit d'instancier la classe **FileWriter** et pointer vers le fichier, et appliquer ensuite la méthode **delete()** à l'objet :

```
import java.io.File;
public class DeleteFile
{
    public static void main( String[] args )
    {
        String filename= "F:/FS/Mes Cours/JAVA/Exp/9 File/newFile1.txt";
        File myFile = new File(filename);
        myFile.delete();
    }
}
```

TP 10 : Gestion des étudiants (Fichier)

Créer une application en Java pour la gestion des étudiants en utilisant un fichier texte «**Etudiant.txt**»:

1. Créer le fichier *Etudiant.txt*
2. Saisir les informations d'un étudiant (Numéro, Nom, Prénom, Email, Note)
3. Afficher la liste des étudiants enregistrés
4. Supprimer le fichier *Etudiant.txt*

Avec l'utilisation d'un switch pour naviguer entre les choix.

Plan du cours :

Développement d'applications en Java

Élément 1 : Fondamentaux et POO en Java

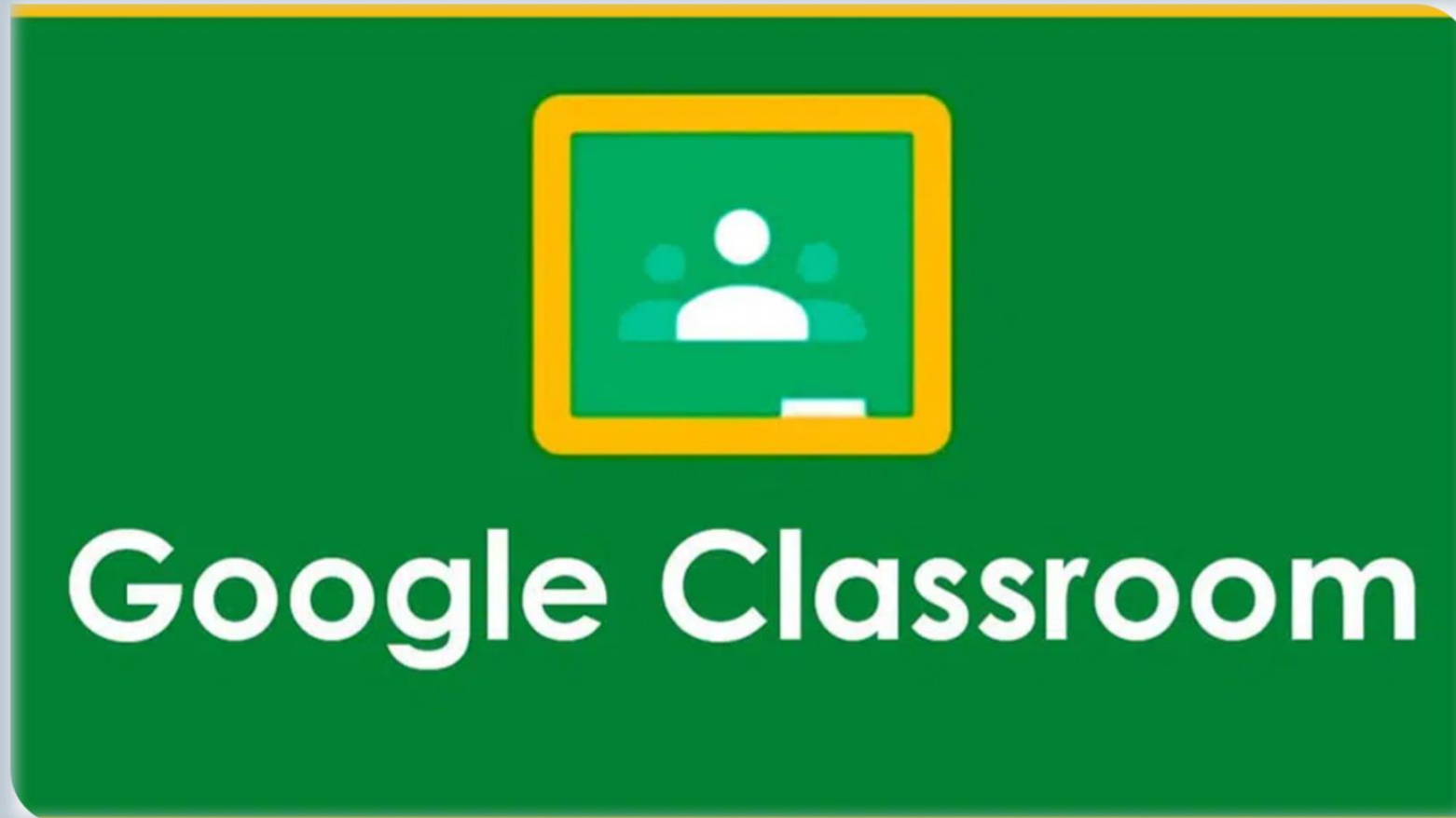
- ✓ Introduction à Java et à l'environnement de développement.
- ✓ Fondamentaux de la programmation Java
- ✓ Programmation Orienté objet en Java
- ✓ Collections Java (List, Set, Map) et leurs applications
- ✓ Manipulation de fichiers
- ✓ Gestion des erreurs et des exceptions

Élément 3 : Application Java en Swing/AWT

- ✓ Concepts des interfaces graphiques (événements, écouteurs).
- ✓ Découverte de Swing et AWT : composants de base (boutons, menus, tableaux).
- ✓ Gestion des événements utilisateur.

Élément 2 : Intégrité de Base de données

- ✓ Connexion d'une application Java à une base de données (JDBC)
- ✓ Requêtes SQL en Java
- ✓ Transactions et sécurité (SQL Injection, PreparedStatement).



Pour toutes informations ou supports pédagogiques, veuillez consulter le **Google Classroom** dédié :

<https://classroom.google.com/c/ODA5ODg2NDcwNTc0?cjc=g7puvgqg>